

RigL on ODT

The Complete Implementation, Line by Line

Every RigL code fragment from the 39-chapter ODT reference, consolidated into build order, with each line explained

Extracted from	ODT_Complete_Reference_v2.pdf (142 pages, 39 chapters)
Source sections	41 RigL sections: 2.1-2.4, the 'Missing RigL Code' section of every chapter, and Chapter 39 in full
Components	7 required + 2 optional (AdamW, Conv1d), in dependency order
Target	STM32 Nucleo-F746ZG, Cortex-M7 at 216 MHz, 320 KB SRAM
Code	C99, every line numbered and explained individually
Also included	An engineering review of the defects found while extracting, and ready-to-compile RigL.h / RigL.c

What This Document Is

In the source reference, the RigL implementation is **not in one place**. It is distributed across 41 sections of 39 chapters: each chapter documents one source file of the ODT library and ends with a 'Missing RigL Code' section describing what that particular file still needs. Chapter 39 then gathers seven of those fragments, but not all of them, and in places states the code differently from the chapter it came from.

This document does the consolidation properly. It collects **every** RigL fragment, orders them by build dependency rather than by file name, explains every line of every fragment individually, and records where each piece came from so you can always go back to the original.

How to read a component chapter

- **Provenance bar** - the grey strip naming the exact section of the source PDF the code came from.
- **Purpose** - what the component does and which other components depend on it.
- **Numbered listing** - the complete code, one line per row.
- **Line-by-line table** - every line of that listing, with what it does and, more usefully, why it is written that way.
- **Notes** - complexity, memory cost on the target MCU, and any defect found in that fragment.

A note on fidelity

The code in this document is the code from the source, transcribed faithfully. Where the source contains an error, the error is reproduced in the listing and then flagged in the line-by-line table and in Chapter 17, rather than silently corrected - you need to know what the original said. Corrected versions are given separately, and always marked as corrections.

NINE DEFECTS WERE FOUND DURING EXTRACTION

The source is a careful document, but consolidating it surfaces problems that are invisible when each chapter is read on its own: two sections give contradictory instructions about when to call `rigLStep()`; one memory figure is wrong by a factor of 250; the DROP and GROW steps as written do not preserve the exact-K conservation the algorithm depends on; and the gradient the GROW step needs is zeroed by another component before GROW can use it. Chapter 17 lists all ten with a proposed fix for each. Read it before you implement.

Build order

1. <code>findAbsKthSmallestActive()</code>	<code>MinMax.c</code>	no dependencies
2. <code>findAbsKthLargestInactive()</code>	<code>MinMax.c</code>	no dependencies
3. <code>weightMask</code> field	<code>Linear.h</code>	enables everything below
+--> 4. <code>mask-aware matmul</code>	<code>Matmul.c</code>	(forward speedup)
+--> 5. <code>mask-aware SGD</code>	<code>Sgd.c</code>	(keeps zeros at zero)
+--> 5b. <code>mask-aware AdamW</code>	<code>AdamW.c</code>	(optional optimiser)
6. <code>rigLStep()</code>	<code>RigL.c</code>	needs 1, 2, 3
7. <code>serializeSparsity()</code>	<code>Serialize.c</code>	persist the mask
8. <code>mask init + UserAPI</code>	<code>UserAPI.c</code>	create the mask
9. <code>training-loop integration</code>	<code>TrainingLoopApi.c</code>	

Figure 0.1 - Dependency order. Implement top to bottom; each step is testable on its own.

Contents

1. The RigL Algorithm	6
1.1 Why this works	6
1.2 The algorithm as stated in the source	6
1.3 The cosine schedule, and why it is shaped that way	7
1.4 Where it stands against dense training	7
2. One RigL Step, Computed by Hand	8
2.1 The toy layer	8
2.2 Step 1 - how many weights to swap	8
2.3 Step 2 - DROP the two weakest active weights	8
2.4 Step 3 - GROW the two most promising inactive weights	9
2.5 Step 4 - clear the gradients of everything still inactive	9
2.6 The layer before and after	9
2.7 The same step with the source's code, and why it differs	10
2.8 The life of one connection, over a whole run	10
3. Prerequisites - the ODT Data Model	12
3.1 tensor_t - the universal container	12
3.2 BOOL tensors and bit packing, worked out	12
3.3 Flat indices - the mapping that must never be wrong	13
3.4 parameter_t - a weight and its gradient, together	13
3.5 layer_t and the config union	14
3.6 Where RigL sits in one training iteration	14
4. What Exists, What Is Missing, and Where Each Piece Lives	15
4.1 Files this touches, and how much of each	15
5. Component 1 - findAbsKthSmallestActive()	17
5.1 Line by line	17
5.2 Worked example	19
5.3 Cost on the target	20
6. Component 2 - findAbsKthLargestInactive()	21
6.1 Line by line - only the differences from Component 1	21
6.2 Cost on the target	23
7. Component 3 - The weightMask Field	24
7.1 Line by line	24
7.2 Two struct definitions, one field	25
7.3 Memory cost	25
8. Component 4 - Mask-Aware Matmul	27
8.1 Line by line	27
8.2 Plumbing the mask into the kernel	28
8.3 Verifying it actually works	28

8.4 The gradient matmul must NOT be masked	28
9. Component 5 - Mask-Aware SGD	30
9.1 Line by line	30
10. Component 5b - Mask-Aware AdamW	32
10.1 Line by line - what differs from SGD	32
10.2 Memory cost of AdamW under sparsity	33
11. Component 6 - rigLStep(), the Algorithm Itself	35
11.1 Line by line	36
11.2 What one call actually does, in order	38
12. Component 7 - Persisting the Mask	40
12.1 Line by line	40
12.2 Wiring it into the tensor record	42
13. Components 8 to 10 - Initialization, Training Loop, Schedules	43
13.1 Component 8 - creating the mask (UserAPI)	43
13.2 Component 9 - the training loop	43
13.3 Component 10 - the two schedules	44
14. Verifying the Implementation	46
14.1 Unit tests, component by component	46
14.2 Integration checks during a real run	46
14.3 Results of a reference simulation	46
15. What You Will See in the Logs	48
15.1 Healthy output	48
15.2 Failure signatures	48
15.3 The three numbers to plot	49
16. An Implementation Plan	50
16.1 Checkpoints where you should stop and verify	50
17. Ten Defects Found While Consolidating	51
17.1 Severity summary	51
17.2 D1 - The thresholds are off by one	51
17.3 D2 - Drop and grow are not disjoint	51
17.4 D3 - The gradients GROW needs may not exist	52
17.5 D4 - Contradictory ordering instructions	52
17.6 D5 - Stale momentum on regrowth	52
17.7 D6 - The AdamW memory figure is not achievable	52
17.8 D7 - Two struct definitions	52
17.9 D8 - malloc on the target	53
17.10 D9 - Two smaller correctness gaps	53
17.11 D10 - The complexity estimate uses the wrong n	53
18. Common Confusions	55
19. Appendix - Consolidated RigL.h and RigL.c	57

19.1 RigL.h	57
19.2 RigL.c	57
20. Appendix - Glossary	60
21. Appendix - Where Every Fragment Came From	62

1. The RigL Algorithm

Source: ODT_Complete_Reference_v2.pdf, sections 2.1, 2.2, 2.3, 2.4 and 33.4

RigL - Rigging the Lottery - trains a sparse network from scratch in a single pass. It keeps a fixed number of active weights throughout training and periodically changes **which** weights are active: it drops the active weights that are smallest in magnitude, and grows the inactive weights whose gradients are largest.

1.1 Why this works

Signal	Used for	Reasoning
$ w[i] $ small, weight active	DROP it	A small weight contributes little to the output, so removing it costs little accuracy
$ g[i] $ large, weight inactive	GROW it	The gradient of an inactive weight says how much the loss WOULD fall if that connection were allowed to move. It is a direct estimate of the value of activating it
K dropped = K grown	Conservation	The active count never changes, so memory and compute stay fixed for the whole run - the property that makes RigL viable on an MCU

THE ONE IDEA

A pruned weight is not dead. Its gradient can still be computed, and that gradient tells you whether it deserves to come back. RigL is the algorithm that acts on this observation: magnitude decides what leaves, gradient decides what enters, and the two are balanced so the sparsity level never moves.

1.2 The algorithm as stated in the source

```

1  RIGL (per sparse layer, every T training steps):
2
3      alpha(t) = (0.3/2) * (1 + cos(pi * t / T_end))    // cosine decay
4      K        = floor( alpha(t) * (1 - s) * N )        // weights to swap
5
6      DROP K active weights with smallest |W|:
7          thresh = K-th smallest |W[i]| in the active set
8          for M[i]=1 and |W[i]| <= thresh: M[i]=0; W[i]=0; G[i]=0
9
10     GROW K inactive weights with largest |G|:
11         thresh = K-th largest |G[i]| in the inactive set
12         for M[i]=0 and |G[i]| >= thresh: M[i]=1; W[i]=0
13
14     Zero the gradients of all remaining inactive weights: G[i]=0

```

Listing 1.1 - The algorithm, transcribed from section 2.2 of the source.

Symbol	Meaning	Typical value for the HAR model
s	Target sparsity	0.9 - nine of every ten weights inactive
N	Total weights in the layer	73,728 for the 1152 x 64 linear layer
(1-s)*N	Active weights - constant for the whole run	7,373
alpha(t)	Fraction of active weights swapped at step t	0.30 falling to 0.00
K	Number swapped this step = alpha * active	2,212 at step 0, 0 after T_end
T	Steps between mask updates	100

Symbol	Meaning	Typical value for the HAR model
T_end	Step after which the mask is frozen	8,280 = 80% of 10,350

1.3 The cosine schedule, and why it is shaped that way

$$\alpha(t) = (\alpha_{\text{init}} / 2) * (1 + \cos(\pi * t / T_{\text{end}}))$$

t = 0 alpha = 0.30 K = 30% of active explore

t = T_end/2 alpha = 0.15 K = 15% of active consolidate

t = T_end alpha = 0.00 K = 0 mask frozen

t > T_end alpha = 0 only weight VALUES change, not positions

The mask must stop moving before training ends. If connectivity were still changing in the final epochs, the newly grown weights - which start at exactly zero - would never get enough updates to become useful, and every swap would cost accuracy instead of buying it. Freezing the mask at 80% of training leaves a fifth of the run to refine a fixed structure.

WHY IT PAIRS WITH A COSINE LEARNING RATE

Both schedules decay together, and they are saying the same thing at different levels. Early on: a high learning rate explores weight VALUES while a high alpha explores weight POSITIONS. Late on: a low learning rate refines values while alpha = 0 holds positions still. Running CosineAnnealingLR and the RigL alpha decay with the same T_max is not a coincidence of convenience - it keeps exploration and refinement aligned in both spaces.

1.4 Where it stands against dense training

Setting	Accuracy	Note
ResNet-50 / ImageNet, dense	76.8%	The reference
RigL, 80% sparse, 1x training	74.6%	2.2 points below dense at one fifth of the weights
RigL, 80% sparse, 5x training	76.4%	Matches dense - the accuracy is recoverable with a longer run
HAR, 90% sparse + FQT	~90-93% (estimate)	The target for this implementation; not yet measured

WHICH OF THESE NUMBERS ARE MEASUREMENTS

The ResNet-50 rows are published results quoted by the source. The HAR row is described in the source as an estimate, and it is repeated here as one. Nothing in this document reports a measurement taken on the STM32 target; when you run it, the numbers in Chapters 14 and 15 are the ones to fill in.

2. One RigL Step, Computed by Hand

Before any C code, work one complete RigL step on a layer small enough to hold in your head. Every number below was computed and checked; you can verify each one with a calculator in about ten minutes. If you understand this chapter, the 51-line `rigLStep()` in Chapter 11 contains no surprises.

2.1 The toy layer

A linear layer with 2 output neurons and 4 inputs - so 8 weights - at 50% sparsity, meaning 4 are active. Weights are stored row-major: row 0 is output neuron 0, row 1 is output neuron 1.

```
weight matrix, [out=2, in=4], row-major

      in0    in1    in2    in3
out0 [  0.80   -0.05   0.00   0.00 ]   flat indices 0 1 2 3
out1 [  0.30   0.00  -0.12   0.00 ]   flat indices 4 5 6 7

mask  [    1     1     0     0
       1     0     1     0 ]

active = {0, 1, 4, 6}    inactive = {2, 3, 5, 7}
```

Figure 2.1 - The toy layer. Inactive weights are exactly 0.0, which is the invariant every component in this document maintains.

Flat index	0	1	2	3	4	5	6	7
weight w	0.80	-0.05	0.00	0.00	0.30	0.00	-0.12	0.00
mask m	1	1	0	0	1	0	1	0
gradient g	0.02	0.85	0.40	-0.03	0.05	0.90	0.01	-0.70

The gradients are the ones the backward pass just produced. Note that **inactive weights have gradients too** - positions 2, 3, 5 and 7 carry 0.40, -0.03, 0.90 and -0.70. That is not an accident of this example; it is the precondition RigL cannot work without, and Chapter 17 (defect D3) is entirely about the ways that precondition gets broken.

2.2 Step 1 - how many weights to swap

```
numActive = 4

alpha      = 0.5      (say we are early in training)
K          = floor(alpha * numActive) = floor(0.5 * 4) = 2
```

So this step will deactivate 2 connections and activate 2 others, leaving the layer with 4 active weights - the same number it started with.

2.3 Step 2 - DROP the two weakest active weights

Collect the magnitudes of the ACTIVE weights only, and sort them:

Rank (0-indexed)	0	1	2	3
 w 	0.05	0.12	0.30	0.80
at flat index	1	6	4	0

```
dropThresh = vals[K] = vals[2] = 0.30
```

```
the two weakest are index 1 (|w| = 0.05) and index 6 (|w| = 0.12)
```


WATCH WHAT THE SOURCE'S COMPARISON DOES HERE

The source tests `|w| <= dropThresh`, i.e. `|w| <= 0.30`. That matches index 1 (0.05), index 6 (0.12) **and index 4 (0.30)** - three weights, not the two we asked for. Using a strict `<` instead matches exactly the two intended. This is defect D1, and it is visible in a layer of eight weights; in a layer of 73,728 it is invisible until you count.

Taking the corrected version, DROP deactivates indices **1 and 6**:

```
mask[1] = 0,  w[1] = 0.0      (was -0.05)
```

```
mask[6] = 0,  w[6] = 0.0      (was -0.12)
```

active is now {0, 4} - only 2 weights, temporarily below target

2.4 Step 3 - GROW the two most promising inactive weights

Now rank the INACTIVE weights by gradient magnitude. The question each gradient answers is: *if this connection were switched on, how fast would the loss fall?*

Rank (0-indexed)	0	1	2	3
g	0.90	0.70	0.40	0.03
at flat index	5	7	2	3

```
growThresh = vals[K] = vals[2] = 0.40
```

the two most promising are index 5 (`|g| = 0.90`) and index 7 (`|g| = 0.70`)

GROW activates indices **5 and 7**, both starting at exactly zero:

```
mask[5] = 1,  w[5] = 0.0
```

```
mask[7] = 1,  w[7] = 0.0
```

active is back to {0, 4, 5, 7} - four weights, conserved

WHY A GROWN WEIGHT STARTS AT ZERO AND NOT AT SOMETHING USEFUL

A newly activated connection has never been trained. Giving it any non-zero value would inject an arbitrary perturbation into a network that is otherwise converging. Starting at zero means the connection contributes nothing on its first forward pass and is then shaped entirely by gradients - it earns its value. The cost is that it needs time to become useful, which is precisely why the mask must stop changing well before training ends.

2.5 Step 4 - clear the gradients of everything still inactive

Positions 1, 2, 3 and 6 are inactive after the swap. Their gradients are zeroed so that this step's values cannot influence the NEXT swap - without this, a weight that had a large gradient once would keep looking attractive forever.

2.6 The layer before and after

Flat index	0	1	2	3	4	5	6	7
mask BEFORE	1	1	0	0	1	0	1	0
w BEFORE	0.80	-0.05	0	0	0.30	0	-0.12	0
mask AFTER	1	0	0	0	1	1	0	1

Flat index	0	1	2	3	4	5	6	7
w AFTER	0.80	0	0	0	0.30	0	0	0

active before: {0, 1, 4, 6} 4 weights
active after : {0, 4, 5, 7} 4 weights <- conserved
two connections moved from weak positions (1, 6)
to positions the gradient says are promising (5, 7)

Notice what survived: index 0 with $|w| = 0.80$ and index 4 with 0.30 were never in danger - large weights are exactly what DROP protects. And notice the price: the trained values -0.05 and -0.12 are gone forever. Every swap discards learned information in exchange for a better position, which is why alpha decays.

2.7 The same step with the source's code, and why it differs

Run the identical data through the source's version - non-strict comparisons, and GROW ranking the post-DROP inactive set - and the result changes:

Step	Corrected version	Source version
DROP candidates	$ w < 0.30 \rightarrow$ indices 1, 6	$ w \leq 0.30 \rightarrow$ indices 1, 6, 4
Weights dropped	2	3 (defect D1)
GROW candidate set	Weights inactive BEFORE the drop: {2, 3, 5, 7}	Weights inactive AFTER the drop: {1, 2, 3, 5, 6, 7} - including the ones just dropped
Ranking by g	0.90 (i5), 0.70 (i7), 0.40 (i2), 0.03 (i3)	0.90 (i5), 0.85 (i1) , 0.70 (i7), 0.40 (i2), ...
Weights grown	indices 5, 7	indices 5, 1 , 7 (defect D2 and D1 together)
Net effect on index 1	Dropped, stays out	Dropped, then IMMEDIATELY regrown in the same step - its trained value -0.05 destroyed for nothing

WHY INDEX 1 IS THE PERFECT ILLUSTRATION OF DEFECT D2

Its weight is tiny (-0.05) so DROP judges it useless, but its gradient is large (0.85) so GROW judges it valuable. Both judgements are reasonable - a weight can be small and still be on a steep part of the loss surface. The published algorithm resolves the conflict by excluding just-dropped weights from growth, so the connection is genuinely retired for at least one interval. The source's version resolves it by resetting a trained weight to zero and putting it back, which is strictly worse than leaving it alone.

2.8 The life of one connection, over a whole run

step	0	mask=1, w=+0.031	born active in the random init mask
step	500	mask=1, w=+0.008	gradients are small; it is shrinking
step	1000	mask=1, w=+0.002	now among the weakest active weights
step	1100	mask=0, w= 0.000	DROPPED. Its value is gone.
step	1100+	mask=0, w= 0.000	still receives a gradient each step, because the weight-gradient matmul stays dense (see Chapter 8)
step	3400	mask=0, $ g =0.21$	the loss surface has changed; this connection now looks valuable
step	3400	mask=1, w= 0.000	REGROWN, starting from zero
step	3500	mask=1, w=-0.014	the optimiser has begun shaping it
step	8000	mask=1, w=-0.190	alpha has reached 0; the mask is frozen and only this value changes

Figure 2.2 - One connection's trajectory. The mask decides whether it participates; the optimiser decides its value; the gradient decides whether it comes back.

THE SENTENCE TO REMEMBER

RigL never asks 'which weights should I delete'. It asks, every 100 steps, 'given what the gradients now say, is the current set of connections still the best set of this size?' - and swaps the few positions where the answer is clearly no. The sparsity level is fixed by you; the PATTERN is learned.

3. Prerequisites - the ODT Data Model

Source: ODT_Complete_Reference_v2.pdf, chapters 3, 4, 6, 10 and 11 of the source

The code in the component chapters manipulates four ODT concepts. Fifteen minutes here makes every later listing readable; skipping it makes the flat-index arithmetic look like magic.

3.1 tensor_t - the universal container

Every array in ODT is a **tensor_t**: a shape, a quantization descriptor saying how the bytes are interpreted, and a flat byte buffer. It is always **one-dimensional in memory**; the shape only says how to read it.

Field	Holds	Why RigL cares
shape	Dimensions, e.g. [64, 1152]	Tells you the row length needed to convert (row, col) to a flat index
quantization	FLOAT32, SYM_INT32, BOOL, ...	Decides whether <code>(float*)t->data</code> is a legal cast. For a mask it must be BOOL
data	A flat byte buffer	The thing every loop in this document walks

calcNumberOfElementsByTensor(t) -> the number of ELEMENTS (not bytes)

FLOAT32 tensor of 8 elements -> 8 elements, 32 bytes

BOOL tensor of 8 elements -> 8 elements, 1 byte (bit-packed)

ELEMENTS, NOT BYTES - AND THE MASK MUST MATCH EXACTLY

The mask and the weight tensor must report the SAME element count, because every consumer indexes both with the same flat index. They occupy wildly different amounts of memory - 4 bytes per weight against 1 bit per mask entry - and confusing the two is how you end up reading past the end of the mask buffer.

3.2 BOOL tensors and bit packing, worked out

A mask stores one bit per weight, eight bits to a byte. The accessor is:

```

1  bool tensorBoolGet(tensor_t *t, size_t i) {
2      uint8_t *d = (uint8_t *)t->data;
3      return (d[i >> 3] >> (i & 7)) & 1;
4  }
5
6  void tensorBoolSet(tensor_t *t, size_t i, bool v) {
7      uint8_t *d = (uint8_t *)t->data;
8      if (v) d[i >> 3] |= (1u << (i & 7));
9      else d[i >> 3] &= ~(1u << (i & 7));
10 }

```

Listing 3.1 - The two accessors every RigL loop calls.

Line	Code	What it does and why
3	<code>d[i >> 3]</code>	<code>i >> 3</code> is <code>i / 8</code> - which byte holds bit <code>i</code> . A shift rather than a division because the compiler generates one instruction either way for unsigned types, and the intent is clearer as a bit operation.
3	<code>>> (i & 7)</code>	<code>i & 7</code> is <code>i % 8</code> - the bit's position within its byte. Shifting it down to position 0...
3	<code>& 1</code>	...and masking off everything above it leaves 0 or 1.

Line	Code	What it does and why
8	<code>d[...] = (1u << (i & 7))</code>	Set: OR in a single 1 bit at that position, leaving the other seven bits of the byte untouched.
9	<code>d[...] &= ~(1u << (i & 7))</code>	Clear: AND with a mask that is all ones except at that position.

Take the toy layer from Chapter 2. Its mask is 8 bits, so exactly one byte:

flat index	7	6	5	4	3	2	1	0	
mask BEFORE	0	1	0	1	0	0	1	1	= 0x53 = 83
mask AFTER	1	0	1	1	0	0	0	1	= 0xB1 = 177

bit `i` sits at position `(i & 7)` of byte `(i >> 3)`, so index 0 is the LOW bit and the byte reads right-to-left when written this way.

`tensorBoolGet(mask, 5)` on 0xB1:

<code>5 >> 3 = 0</code>	-> byte 0, which is 0xB1 = 1011 0001
<code>5 & 7 = 5</code>	-> shift right by 5: 0000 0101
<code>& 1</code>	-> 1, so weight 5 is ACTIVE

Figure 3.1 - The toy layer's mask as actual bits.

Layer	Weights	Mask bytes	Weight bytes (FP32)	Overhead
Toy example	8	1	32	3.1%
HAR 1152 x 64	73,728	9,216	294,912	3.1%
Output 64 x 6	384	48	1,536	3.1%

The overhead is always 1 bit per 32, i.e. 3.125%, whatever the layer size - a useful constant to quote when someone asks what the mask costs.

3.3 Flat indices - the mapping that must never be wrong

The weight matrix is logically 2-D and physically 1-D. Row-major means row 0 comes first, complete, then row 1:

flatIdx = rowIndex * columnsPerRow + columnIndex	
toy layer: 2 outputs x 4 inputs, so columnsPerRow = 4	
(out 0, in 0) -> 0*4 + 0 = 0	(out 1, in 0) -> 1*4 + 0 = 4
(out 0, in 1) -> 0*4 + 1 = 1	(out 1, in 1) -> 1*4 + 1 = 5
(out 0, in 2) -> 0*4 + 2 = 2	(out 1, in 2) -> 1*4 + 2 = 6
(out 0, in 3) -> 0*4 + 3 = 3	(out 1, in 3) -> 1*4 + 3 = 7

In the mask-aware matmul of Chapter 8 this appears as `flatIdx = rowIndex * aColumns + i`, where `rowIndex` is the output neuron being computed and `i` walks the inputs. It is the same formula. The reason it deserves this much attention is that a mistake here does not crash - it skips the wrong weights, and the model trains happily to a wrong answer.

THE FIVE-MINUTE CHECK THAT PROVES YOUR INDEX IS RIGHT

Set every inactive weight to 0.0, then run the ORDINARY dense matmul and compare against your masked one. They must agree bit for bit, because skipping a weight and multiplying by a stored zero are the same arithmetic. If they disagree, your flat index does not match the mask's ordering. This one test is worth more than any amount of staring at the formula.

3.4 parameter_t - a weight and its gradient, together

```

typedef struct {
    tensor_t *param;    /* the weights themselves */
    tensor_t *grad;     /* dL/dW, same shape, filled by backward() */
} parameter_t;

```

rigLStep() needs both halves: `param` for the DROP decision and `grad` for GROW. They have identical shapes and element counts, so one flat index addresses the weight, its gradient and its mask bit - which is what makes the loops in Chapter 11 as simple as they are.

3.5 layer_t and the config union

```

layer_t { layerType_t type; layerConfig_t *config; ... }
layerConfig_t is a UNION: .linear, .conv1d, .relu, ...

so: model[l]->config->linear is valid ONLY when
    model[l]->type == LINEAR

```

WHY THE TYPE CHECK COMES FIRST IN RIGLSTEP()

`config` is a union, so reading `->linear` on a Conv1d layer does not fail - it reinterprets whatever bytes are there as a `linearConfig_t` and hands you a garbage `weightMask` pointer. The line `if (model[l]->type != LINEAR) continue;` is not a filter for tidiness; it is what stops the function from dereferencing nonsense.

3.6 Where RigL sits in one training iteration

```

getBatch()
|
forward()      <- masked matmul: skips inactive weights
|
loss
|
backward()     <- weight-gradient matmul: DENSE, so that
|               inactive weights still receive gradients
|
rigLStep()     <- every 100 steps: DROP + GROW using those
|               gradients, before anything clears them
|
optimizer.step() <- masked update: active weights move,
|               inactive ones are forced to exactly 0
next iteration

```

Figure 3.2 - The order of operations. Two things in this diagram are load-bearing: `backward()` must be dense, and `rigLStep()` must run before the optimiser. Chapter 17 explains what breaks otherwise.

4. What Exists, What Is Missing, and Where Each Piece Lives

Source: ODT_Complete_Reference_v2.pdf, section 2.4 ('RigL: ODT Gap') and the 'Missing RigL Code' section of all 39 chapters

Before writing anything, it is worth knowing exactly how much of RigL the ODT library already provides. The answer is: the primitives exist, the algorithm does not.

Already in ODT	What it gives you
BOOL tensor type	A bit-packed tensor - the mask storage format, 1 bit per weight rather than 1 byte
tensorBoolGet / tensorBoolSet	Read and write one mask bit by flat index
bernoulliFillMask	Fill a BOOL tensor with independent Bernoulli draws - the random initial mask
calcNumberOfElementsByTensor	Element count, used by every loop in this document
XorShift32 RNG with seeding	Reproducible masks and reproducible stochastic rounding
Cosine LR scheduler	The same cosine math the alpha schedule needs

Missing - must be built	File	This document
findAbsKthSmallestActive()	MinMax.c	Chapter 5
findAbsKthLargestInactive()	MinMax.c	Chapter 6
weightMask field in linearConfig_t	Linear.h	Chapter 7
Mask-aware inner loop	Matmul.c	Chapter 8
Mask-aware parameter update	Sgd.c	Chapter 9
Mask-aware update with moment clearing	AdamW.c	Chapter 10
rigLStep()	RigL.h / RigL.c (new)	Chapter 11
serializeSparsity() / deserializeSparsity()	Serialize.c	Chapter 12
Mask creation for sparse layers	UserAPI.c	Chapter 13
rigLStep() call site and config fields	TrainingLoopApi.c	Chapter 13

SEVEN, EIGHT OR TEN COMPONENTS?

The source says seven. That count omits `deserializeSparsity()` - without which a saved mask cannot be loaded back, so checkpointing is only half implemented - and it omits the UserAPI and training-loop integration described in chapters 38 and 37, without which nothing ever calls `rigLStep()`. Counting honestly there are ten pieces of work, which is why this document has ten component chapters rather than seven.

4.1 Files this touches, and how much of each

File	Change	Size of change	Risk if wrong
MinMax.c / .h	Two new functions	~60 lines	Wrong threshold - drops or grows the wrong count
Linear.h	One struct field, one initialiser line	2 lines	None if NULL-defaulted; dense layers keep working
Matmul.c	One conditional in the hot loop	~4 lines	Silently wrong outputs if the flat index is miscomputed
Sgd.c	One conditional in the update loop	~5 lines	Inactive weights drift away from zero and sparsity is fake

File	Change	Size of change	Risk if wrong
AdamW.c	One conditional plus moment clearing	~8 lines	Stale moments cause a bad first step after a grow
RigL.c / .h	New file	~50 lines	The algorithm itself
Serialize.c	Two functions	~25 lines	Mask lost at checkpoint - the model reloads dense
UserAPI.c	Mask allocation for sparse specs	~8 lines	No mask means rigLStep() silently skips every layer
TrainingLoopApi.c	Config fields and one call	~10 lines	The mask never updates; you get static random sparsity

5. Component 1 - findAbsKthSmallestActive()

Source: ODT_Complete_Reference_v2.pdf, section 7.4 (primary) and section 39.2 (variant)

Purpose. Find the K-th smallest absolute weight value among the **active** weights of a layer. This single number is the DROP threshold: every active weight whose magnitude is at or below it will be deactivated.

Depends on: nothing. **Used by:** rigLStep() (Chapter 11). This is the first thing to implement and the easiest to unit-test.

```

1  float findAbsKthSmallestActive(tensor_t *weights,
2                                tensor_t *mask,
3                                size_t K) {
4      size_t n = calcNumberOfElementsByTensor(weights);
5      float *w = (float *)weights->data;
6
7      /* Pass 1: how many weights are currently active? */
8      size_t count = 0;
9      for (size_t i = 0; i < n; i++)
10         if (tensorBoolGet(mask, i)) count++;
11
12     if (K >= count) return 1e38f; /* keep all: threshold = +inf */
13
14     float *vals = malloc(count * sizeof(float));
15     if (!vals) { exit(1); }
16
17     /* Pass 2: gather |w| of the active weights only. */
18     size_t idx = 0;
19     for (size_t i = 0; i < n; i++)
20         if (tensorBoolGet(mask, i)) vals[idx++] = fabsf(w[i]);
21
22     /* Partial selection sort: sorts only the first K+1 positions. */
23     for (size_t i = 0; i <= K && i < count; i++) {
24         size_t minIdx = i;
25         for (size_t j = i + 1; j < count; j++)
26             if (vals[j] < vals[minIdx]) minIdx = j;
27         float tmp = vals[i];
28         vals[i] = vals[minIdx];
29         vals[minIdx] = tmp;
30     }
31
32     float thresh = vals[K < count ? K : count - 1];
33     free(vals);
34     return thresh;
35 }

```

Listing 3.1 - findAbsKthSmallestActive(), transcribed from section 7.4.

5.1 Line by line

Line	Code	What it does and why
1	float findAbsKthSmallestActive(tensor_t *weights,	Does: declares a function returning a float threshold. Why a float and not a list of indices: returning a threshold makes the function pure and stateless - it allocates nothing the caller must free, and the caller decides what to do with it. A function that returned 'the K indices to drop' would need a buffer, an ownership convention, and a second pass anyway.
2	tensor_t *mask,	Does: the mask defining which weights count. Why it is a parameter rather than read from the layer: this keeps the function independent of <code>linearConfig_t</code> , so it can be unit tested with two bare tensors and no model - which is exactly how Chapter 14 tests it.

Line	Code	What it does and why
3	<code>size_t K) {</code>	Does: how many weights the caller intends to drop. Why `size_t`: it indexes into the array on line 31, and a signed type would invite a negative K that indexes backwards.
4	<code>size_t n = calcNumberOfElements ByTensor(weights);</code>	Does: the total element count, active and inactive. Why total and not active: the loops on lines 9 and 18 walk the whole tensor, because the mask is a bitmap with no index list. Careful: ELEMENTS, not bytes - the mask must report the same number.
5	<code>float *w = (float *)weights->data;</code>	Does: a raw float view, bypassing the tensor accessors. Why: this pointer is dereferenced up to n times in the gather loop; a function call per element would dominate the cost. If wrong: valid only for FLOAT32. A SYM_INT32 tensor holds int32 mantissas, and reinterpreting those bits as floats yields magnitudes unrelated to the real weights. The fix is to reconstruct <code>mantissa * scale</code> first - a planned enhancement in the source, not implemented.
7	<code>/* Pass 1: how many weights are currently active? */</code>	Does: nothing at runtime. Why it earns its place: this function makes three passes over the data and a reader needs to know which one they are in.
8	<code>size_t count = 0;</code>	Does: initialises the active counter. Why it must be counted at all: nothing in ODT stores the active count; the mask is the sole record, so it is recomputed on every call.
9	<code>for (size_t i = 0; i < n; i++)</code>	Does: the counting scan. Why over n and not over some smaller set: there is no list of active positions to iterate - finding them IS this loop.
10	<code>if (tensorBoolGet(mask, i)) count++;</code>	Does: tests one mask bit; increments if set. Cost: <code>tensorBoolGet</code> is a load, a shift and an AND - about 3-4 cycles on a Cortex-M7, so this pass costs roughly 4n cycles. For the HAR layer, about 300,000 cycles, or 1.4 ms at 216 MHz.
12	<code>if (K >= count) return 1e38f;</code>	Does: handles the case where the caller asks to drop at least as many weights as are active. Why 1e38 specifically: it is a finite float just under FLT_MAX (3.4e38), so it compares cleanly and never becomes an infinity that could propagate as a NaN in later arithmetic. Every active weight satisfies <code> w <= 1e38</code> , so all of them drop - the correct reading of 'you asked for more than exist'. Second job: it guarantees <code>K < count</code> for line 31, which would otherwise index past the end of the buffer.
14	<code>float *vals = malloc(count * sizeof(float));</code>	Does: allocates a scratch buffer holding one float per active weight. Why heap: <code>count</code> is a runtime value, so a fixed-size array would have to be sized for the worst case. Why this is the worst line in the file on an MCU: 29 KB here, and 259 KB in the sibling function, on a 320 KB device with no MMU, allocated and freed every 100 steps. Repeated allocation of differently-sized blocks fragments the heap; the failure mode is a NULL return hours into a run. Defect D8.
15	<code>if (!vals) { exit(1); }</code>	Does: aborts on allocation failure. Why it is unacceptable on the target: there is no operating system to exit to. On bare metal <code>exit()</code> either hangs in a loop or invokes undefined behaviour, and either way a training run dies with no diagnostic. Return an error and let the caller skip this layer's RigL step - one missed swap is harmless.
17	<code>size_t idx = 0;</code>	Does: the write cursor into the scratch buffer. Why a separate counter: the read index <code>i</code> walks all n positions while the write index advances only on active ones, so the two cannot be the same variable.
18	<code>for (size_t i = 0; i < n; i++)</code>	Does: the gather scan - the second full pass. Why not fused with pass 1: the buffer cannot be allocated until its size is known, and its size is what pass 1 computes. Fusing them would need a growable buffer or an upper-bound allocation of n floats, which for the HAR layer is 288 KB - worse than the problem it solves.

Line	Code	What it does and why
19	<pre>if (tensorBoolGet(mask, i)) vals[idx++] = fabsf(w[i]);</pre>	Does: copies <code> w </code> of each active weight into the dense buffer. Why absolute value: RigL ranks by magnitude - a weight of -0.8 is just as influential as +0.8. Why <code>`fabsf`</code> and not <code>`fabs`</code>: the float version. <code>fabsf</code> takes a double, so it promotes, computes in double, and demotes - on an M7 with a single-precision FPU that means a software double routine, costing tens of cycles per element instead of one. Postcondition: <code>idx == count</code> .
21	<pre>/* Partial selection sort: first K+1 positions. */</pre>	Does: nothing at runtime. Why it matters: without this comment a reader assumes the loop below is a broken full sort.
22	<pre>for (size_t i = 0; i <= K && i < count; i++) {</pre>	Does: the outer loop of a partial selection sort, running K+1 times. Why <code>`<= K`</code> and not <code>`< K`</code>: positions 0..K must all be settled, because line 31 reads <code>vals[K]</code> . Why partial at all: a full sort is $O(\text{count}^2)$ here; stopping early makes it $O(\text{count} \times K)$. For the HAR layer that is 16.3 M comparisons instead of 54 M. The <code>`i < count`</code> term is redundant given line 12, but survives a future change to that guard.
23	<pre>size_t minIdx = i;</pre>	Does: assumes the current position holds the smallest remaining value. Why: selection sort's invariant - everything before <code>i</code> is already in its final place, so the search starts at <code>i</code> .
24	<pre>for (size_t j = i + 1; j < count; j++)</pre>	Does: scans the unsorted tail. Why from <code>`i+1`</code>: positions at or below <code>i</code> are settled. Cost: this is the hot loop; it executes about <code>count × K</code> times in total and is where essentially all the function's time goes.
25	<pre>if (vals[j] < vals[minIdx]) minIdx = j;</pre>	Does: tracks the index of the smallest value seen. Why <code>`<`</code> and not <code>`<=`</code>: with <code><=</code> the index would keep moving among equal values for no benefit; either is correct, <code><</code> does less work. Note: this is the ONE line that differs between this function and its sibling, where it becomes <code>></code> .
26	<pre>float tmp = vals[i];</pre>	Does: the first step of a three-line swap. Why a manual swap: C has no swap operator, and a memcpy of four bytes would be slower than three register moves.
27	<pre>vals[i] = vals[minIdx];</pre>	Does: moves the smallest remaining value into position <code>i</code> .
28	<pre>vals[minIdx] = tmp;</pre>	Does: completes the swap. Postcondition after the outer loop: positions 0..K hold the K+1 smallest magnitudes in ascending order; positions K+1.. are in arbitrary order, which is fine because they are never read.
31	<pre>float thresh = vals[K < count ? K : count - 1];</pre>	Does: reads the K-th smallest magnitude, zero-indexed - so actually the (K+1)-th smallest value. Toy layer: <code>vals[2] = 0.30</code> . Why the ternary: redundant given line 12, but it makes the function safe on its own terms rather than relying on a guard forty lines away. Where defect D1 lives: paired with the <code><=</code> in <code>rigLStep()</code> , this selects K+1 weights. Either return <code>vals[K-1]</code> and keep <code><=</code> , or keep this and use a strict <code><</code> .
32	<pre>free(vals);</pre>	Does: releases the scratch buffer. Why here and not after the return: it must precede the return, and <code>thresh</code> was copied out on line 31 precisely so that this is possible. Every path except the <code>exit(1)</code> frees, so there is no leak.
33	<pre>return thresh;</pre>	Does: hands the caller a single float. What the caller does with it: compares every active weight against it. The function itself never touches the mask - it is read-only over its inputs, which is what makes it safe to call twice or to test in isolation.
34	<pre>}</pre>	Does: end of function. Note what is NOT here: no mask mutation, no weight mutation, no logging, no global state. That purity is why this is the first component to implement and the easiest to trust.

Table 5.1 - Every line of `findAbsKthSmallestActive()`. Toy-layer values refer to the 8-weight example of Chapter 2.

5.2 Worked example

The source gives this example in section 39.2. It is worth working through carefully, because it exposes a defect.

`active |w| = [0.01, 0.05, 0.03, 0.12, 0.08],    K = 2`

`after the partial sort: [0.01, 0.03, 0.05, ...]`

`thresh = vals[2] = 0.05`

`the source then states: 'weights with |w| <= 0.05 are dropped:`

`0.01 and 0.03. Exactly 2 weights dropped = K.'`

THE EXAMPLE UNDERCOUNTS BY ONE

The condition is `|w| <= 0.05` and the value 0.05 is itself in the active set - so THREE weights match: 0.01, 0.03 and 0.05. To drop exactly K you need either `thresh = vals[K-1]` with `<=`, or `thresh = vals[K]` with a strict `<`. As written, DROP removes K+1 weights. The GROW step has the mirror-image error, so the two happen to cancel and total sparsity is preserved - but the swap size is not the K you asked for, and with duplicate magnitudes the cancellation stops being exact. Chapter 17, defect D1.

5.3 Cost on the target

Quantity	Value for the HAR 1152 x 64 layer at 90% sparsity
n (all weights)	73,728
count (active)	7,373
K at alpha = 0.3	2,212
Comparisons, O(count*K)	7,373 x 2,212 = 16.3 M
Scratch memory	7,373 x 4 B = 29 KB on the heap
Frequency	Once per sparse layer, every 100 steps

THE SOURCE'S COMPLEXITY ESTIMATE IS OFF

Section 7.4 estimates the cost as `n * K` and gives $8192 \times 819 = 6.7$ M comparisons for a 128 x 64 layer. The sort runs over the ACTIVE values only, so the true cost is `count * K`, which at 90% sparsity is an order of magnitude smaller than the n-based figure. The estimate is conservative in the safe direction, but if you are budgeting cycles, use count, not n. Chapter 17, defect D10.

MALLOC ON A CORTEX-M7

29 KB from the heap, allocated and freed every 100 steps, for every sparse layer, on a device with 320 KB of SRAM and no MMU. Repeated allocation of differently-sized blocks fragments the heap, and the failure mode is a NULL return at some unpredictable point hours into training. Two fixes: allocate one scratch buffer of the largest layer size ONCE at initialisation and reuse it, or replace the gather-and-sort with a two-pass histogram over magnitudes that needs only a fixed-size bin array. The first fix is a ten-line change and is what this document recommends.

6. Component 2 - findAbsKthLargestInactive()

Source: ODT_Complete_Reference_v2.pdf, section 7.4 (primary) and section 39.3 (variant)

Purpose. Find the K-th largest absolute **gradient** among the **inactive** weights. This is the GROW threshold: every inactive weight whose gradient magnitude is at or above it will be activated. It is the mirror image of Component 1 - inactive instead of active, gradients instead of weights, largest instead of smallest.

```

1  float findAbsKthLargestInactive(tensor_t *grads,
2                                tensor_t *mask,
3                                size_t K) {
4      size_t n = calcNumberOfElementsByTensor(grads);
5      float *g = (float *)grads->data;
6
7      size_t count = 0;
8      for (size_t i = 0; i < n; i++)
9          if (!tensorBoolGet(mask, i)) count++;
10
11     if (K >= count) return 0.0f;    /* grow all: threshold = 0 */
12
13     float *vals = malloc(count * sizeof(float));
14     if (!vals) { exit(1); }
15
16     size_t idx = 0;
17     for (size_t i = 0; i < n; i++)
18         if (!tensorBoolGet(mask, i)) vals[idx++] = fabsf(g[i]);
19
20     /* Partial selection sort, DESCENDING this time. */
21     for (size_t i = 0; i <= K && i < count; i++) {
22         size_t maxIdx = i;
23         for (size_t j = i + 1; j < count; j++)
24             if (vals[j] > vals[maxIdx]) maxIdx = j;
25         float tmp = vals[i];
26         vals[i] = vals[maxIdx];
27         vals[maxIdx] = tmp;
28     }
29
30     float thresh = vals[K < count ? K : count - 1];
31     free(vals);
32     return thresh;
33 }
```

Listing 4.1 - findAbsKthLargestInactive(), from section 7.4.

6.1 Line by line - only the differences from Component 1

Line	Code	What it does and why
1	float findAbsKthLargestInactive (tensor_t *grads,	Does: declares the GROW-side selector. Why the first argument is GRADIENTS, not weights: this is the conceptual heart of RigL. An inactive weight is exactly zero, so its magnitude carries no information at all. Its GRADIENT, by contrast, is the derivative of the loss with respect to that connection - a direct estimate of how much the loss would fall if it were switched on. Ranking by it is what makes RigL better than random regrowth.
2	tensor_t *mask,	Does: the same mask as Component 1, but read with the opposite polarity. Why one mask serves both: a single bitmap defines both sets - active is bit 1, inactive is bit 0 - so there is nothing to keep in sync.

Line	Code	What it does and why
3	size_t K) {	Does: how many connections to activate. Why it should equal the number actually dropped, not the nominal K: if DROP removed K+1 because of the tie behaviour in Component 1, growing only K leaves the layer one connection sparser than it started. The fix in Appendix A passes dropped here for exactly this reason.
4	size_t n = calcNumberOfElements ByTensor(grads);	Does: total elements. Why from the gradient tensor: it has the same shape as the weights, so either would do; taking it from the tensor actually being read keeps the function self-consistent.
5	float *g = (float *)grads->data;	Does: raw float view of the gradients. If wrong: the FQT caveat is sharper here than in Component 1. Gradient tensors in ODT are the ones most likely to be SYM_INT32, because that is the point of fully quantized training - so this cast is the first thing to revisit when FQT is switched on. Defect D3b.
7	size_t count = 0;	Does: initialises the INACTIVE counter. Scale note: at 90% sparsity this will reach about 66,355 for the HAR layer - nine times what Component 1 counts, which is what makes this the expensive half.
8	for (size_t i = 0; i < n; i++)	Does: the counting scan, identical in structure to Component 1.
9	if (!tensorBoolGet(mask, i)) count++;	Does: counts CLEAR bits. The one character that matters: the ! . Copying this function from its sibling and forgetting the negation gives you a threshold computed over the wrong population - and since both functions return a plausible small float, nothing about the result looks wrong.
11	if (K >= count) return 0.0f;	Does: the degenerate guard. Why 0.0 and not 1e38 as in Component 1: because the GROW comparison is g >= thresh . A threshold of zero matches every inactive weight, so all of them grow - the correct meaning of 'you asked for more than exist'. If wrong: returning 1e38 here, by false symmetry with Component 1, would grow NOTHING while DROP still removed K - and the layer would get sparser at every RigL step until it was empty. This is the most instructive asymmetry in the whole implementation.
13	float *vals = malloc(count * sizeof(float));	Does: allocates the scratch buffer. Why this is the blocker, not merely a smell: count is the INACTIVE population, so for the HAR layer this asks for 66,355 x 4 = 259 KB on a device with 320 KB of SRAM total, while weights, gradients and activations are all resident. It will not allocate. Component 1's 29 KB might; this will not. Defect D8, and the histogram replacement is in Chapter 17.
14	if (!vals) { exit(1); }	Does: aborts on failure - which, given the line above, is the path you should expect to take on the target. Better: return 1e38f, which grows nothing and lets this RigL step be skipped harmlessly, and log the failure.
16-18	gather g of inactive weights into vals	Does: the second pass, mirroring Component 1 but selecting the complementary set. Where defect D3 becomes visible: if the inactive gradients were never computed - because the weight-gradient matmul was masked - or were zeroed by the optimiser or by the previous rigLStep(), then every value written here is 0.0. The function still returns a number, the threshold is 0.0, and GROW then matches every inactive weight in index order. Nothing crashes; RigL simply stops being RigL.
20	/* Partial selection sort, DESCENDING this time. */	Does: nothing at runtime, but flags the single structural difference from Component 1.
21	for (size_t i = 0; i <= K && i < count; i++) {	Does: the same partial sort loop, K+1 iterations. Cost: count x K here is 66,355 x 2,212 = 147 M comparisons for the HAR layer - nine times Component 1, and the reason a cheaper selection method matters more on this side.
22	size_t maxIdx = i;	Does: tracks the largest remaining value rather than the smallest. Why the rename from `minIdx`: it is not cosmetic - a reader scanning for the difference between these two functions should find it in the identifier, not only in the comparison.

Line	Code	What it does and why
23-24	<pre>if (vals[j] > vals[maxIdx]) maxIdx = j;</pre>	Does: the comparison flipped to <code>></code> . This is the entire algorithmic difference between Components 1 and 2. Everything else - the counting pass, the gather, the swap, the guards - is the same code with the mask polarity inverted.
25-27	<pre>swap vals[i] and vals[maxIdx]</pre>	Does: moves the largest remaining value into position <code>i</code> . Postcondition: positions 0..K hold the K+1 largest gradient magnitudes in DESCENDING order.
29	<pre>float thresh = vals[K < count ? K : count - 1];</pre>	Does: the K-th largest magnitude, zero-indexed. Toy layer: <code>vals[2] = 0.40</code> . Defect D1, mirrored: paired with a non-strict <code>>=</code> in <code>rigLStep()</code> this activates K+1 weights. Since DROP also over-selects by one, the two cancel and the active count survives - by luck rather than design, and the luck runs out when magnitudes tie unevenly.
30-31	<pre>free(vals); return thresh;</pre>	Does: releases the buffer and returns the threshold. Same purity as Component 1: no mask mutation, no global state, testable with four numbers in an array.

Table 6.1 - Every line of `findAbsKthLargestInactive()`, with the differences from Component 1 called out.

WHY THE TWO GUARDS RETURN OPPOSITE EXTREMES

This asymmetry looks like an inconsistency and is in fact the correct design. The DROP guard returns `+inf` so that `|w| <= inf` matches everything; the GROW guard returns 0 so that `|g| >= 0` matches everything. Both mean 'process all candidates'. If you copy one function to make the other - which is the obvious way to write them - this is the line to check first.

6.2 Cost on the target

Quantity	Value for the HAR 1152 x 64 layer at 90% sparsity
count (inactive)	66,355 - nine times Component 1
K at alpha = 0.3	2,212
Comparisons, $O(\text{count} \times K)$	$66,355 \times 2,212 = 147 \text{ M}$
Scratch memory	$66,355 \times 4 \text{ B} = 259 \text{ KB}$ on the heap
Verdict	Infeasible as written on a 320 KB device

THIS IS THE COMPONENT THAT WILL NOT FIT

259 KB of scratch heap on a device with 320 KB of SRAM total, while the weights, gradients, activations and optimiser state are also resident, is not going to allocate. Component 2 is where the malloc approach stops being merely inelegant and becomes a blocker. The histogram alternative - two passes over the gradients accumulating counts into, say, 256 magnitude bins, then a partial scan to find the bin containing the K-th largest - needs about 1 KB and no allocation at all, at the cost of a threshold that is approximate to within one bin width. For a swap decision that is re-made every 100 steps, an approximate threshold is entirely acceptable. Chapter 17, defect D8.

7. Component 3 - The weightMask Field

Source: ODT_Complete_Reference_v2.pdf, section 11.4 (primary) and section 39.4 (variant)

Purpose. Give a linear layer somewhere to keep its mask. This is two lines of code and it is the keystone of the whole implementation: the matmul, the optimiser and rigLStep() all reach for this one field. **Depends on:** nothing. **Everything else depends on it.**

```

1  /* Linear.h - add one field to the config struct */
2  typedef struct linearConfig {
3      parameter_t *weights;
4      parameter_t *bias;
5      bool hasBias;
6      arithmetic_t arithType;
7      tensor_t *weightMask; /* NEW: NULL = dense, non-NULL = RigL */
8  } linearConfig_t;
9
10 /* Linear.c - in linearInitConfig() */
11 cfg->weightMask = NULL; /* dense by default */
12
13 /* To enable RigL on a layer, at model-build time: */
14 tensor_t *mask = createBoolTensor(outFeatures * inFeatures);
15 bernoulliFillMask(mask, 1.0f - targetSparsity); /* 0.1 => 90% sparse */
16 cfg->weightMask = mask;

```

Listing 5.1 - The weightMask field and its initialisation.

7.1 Line by line

Line	Code	What it does and why
2	typedef struct linearConfig {	Does: opens the existing config struct. Warning: do not retype this struct from either listing in the source - sections 11.4 and 39.4 show different field lists and at least one is wrong. Open the real header and add the single field.
3-6	weights, bias, hasBias, arithType (existing fields)	Does: nothing new; shown for context. Why they matter here: weights is a parameter_t , which bundles the value tensor with its gradient tensor. rigLStep() reaches both through this one field, which is why the mask can live beside it and share a flat index with both.
7	tensor_t *weightMask;	Does: adds a pointer to the layer's BOOL mask tensor. Why a pointer and not an embedded tensor_t: a dense layer then pays 4 bytes for a null pointer instead of carrying an unused structure, and the mask can be allocated, replaced or freed independently of the config. Why in linearConfig_t and not layer_t: only linear layers support RigL in this implementation; putting it in the generic layer struct would imply conv and pooling layers have masks too.
8	} linearConfig_t;	Does: closes the struct. Binary compatibility note: adding a field changes sizeof and every offset after it. Anything that serializes this struct raw, or was compiled against the old header, must be rebuilt.
11	cfg->weightMask = NULL;	Does: the default, set in linearInitConfig() . Why this is the single most important line in the component: every consumer tests if (mask != NULL) first, so NULL means 'behave exactly as before'. That is what makes this change additive - no existing model, test or checkpoint changes behaviour. If wrong: omit it and the pointer is whatever was on the heap. The matmul then calls tensorBoolGet on garbage and you get a hard fault at a random point, in code that looks unrelated.

Line	Code	What it does and why
14	<code>tensor_t *mask = createBoolTensor(outFeatures * inFeatures);</code>	Does: allocates a bit-packed mask with one bit per weight. Why this exact size: it must equal <code>calcNumberOfElementsByTensor(weights)</code> exactly, because the matmul, the optimiser and <code>rigLStep()</code> all index the mask with the weight's flat index. Note the bias is NOT masked - it is a separate tensor and biases are left dense. Concretely: $64 \times 1152 = 73,728$ bits = 9,216 bytes.
15	<code>bernoulliFillMask(mask, 1.0f - targetSparsity);</code>	Does: fills the mask with independent Bernoulli draws. The inversion trap: the argument is P(ACTIVE). For 90% sparsity you pass 0.1, not 0.9. Passing 0.9 gives a 90% DENSE model that trains correctly and is nine times slower than you expected - a bug with no error message. Why the realised count is not exact: independent draws make it binomial. For 73,728 weights at $p = 0.1$ the mean is 7,373 with a standard deviation of about 81, so 7,412 active is normal, not a fault. If you need exactly the target, shuffle an array with exactly that many ones instead.
16	<code>cfg->weightMask = mask;</code>	Does: attaches the mask, switching the layer into sparse mode. What is missing here: two things. First, <code>zeroInactiveWeights()</code> - <code>bernoulliFillMask</code> sets bits but does not touch the weights, so until you zero them the inactive positions still hold their random initial values. Section 38.4 includes that call and section 11.4 does not. Second, ownership: nothing records who frees this tensor. <code>linearConfig_t</code> already has an <code>ownsQuantizations</code> flag for exactly this kind of question, and the mask needs the same treatment or it leaks on every model teardown.

Table 7.1 - Every line of the weightMask change. Two lines of code, and both of the notes above are things the source leaves unstated.

TWO THINGS THE SOURCE LEAVES UNSTATED

OWNERSHIP: nothing says who frees this tensor. `linearConfig_t` already has an `ownsQuantizations` flag for exactly this kind of question, and the mask needs the same treatment or it leaks on every model teardown. ENFORCEMENT AT INIT: `bernoulliFillMask` sets the mask bits but does not touch the weights, so immediately after initialisation 90% of the weights are inactive yet still hold their random initial values. Section 38.4 calls `zeroInactiveWeights()` to fix this and section 11.4 does not mention it. If you skip it, the first forward pass is dense-valued but mask-skipped, and your loss curve starts from the wrong place.

7.2 Two struct definitions, one field

Sections 11.4 and 39.4 both show `linearConfig_t`, and they disagree about the other fields:

Section 11.4 shows	Section 39.4 shows
weights, bias, hasBias, arithType	weights, bias, four separate arithmetic_t fields, two quantization_t pointers, two outputMode_t fields, ownsQuantizations

Only the second is plausible for a library that supports FQT - the four arithmetic fields correspond to the forward, weight-gradient, bias-gradient and loss-propagation paths. The lesson for you is practical: **do not retype the struct from either listing**. Open `Linear.h`, find `linearConfig_t`, and add the one field. Chapter 17, defect D7.

7.3 Memory cost

Item	Bytes	Note
Mask, 1152 x 64 layer	9,216	1 bit per weight, bit-packed
Mask, 64 x 6 output layer	48	Usually left dense - see below
Weight tensor, unchanged	294,912	Still stored densely, with zeros in the inactive positions
Overhead of masking	3.1%	$9,216 / 294,912$

SPARSE HERE MEANS SKIPPED, NOT ABSENT

The weight tensor keeps its full dense allocation; the mask only says which entries participate. So RigL as implemented here buys COMPUTE, not weight MEMORY - the 294 KB tensor is still 294 KB. Getting the memory back needs a compressed format such as CSR, which costs indices and destroys the $O(1)$ random access that the mask-aware matmul relies on. For a device where the bottleneck is cycles and energy rather than flash, dense-plus-mask is the right trade, but be clear about which resource you are actually saving before quoting a 10x figure to anyone.

DO NOT SPARSIFY THE LAST LAYER

The 64 x 6 classifier holds 384 weights - 0.5% of the model. Making it 90% sparse saves nothing measurable and removes connections to individual output classes, which is exactly where damage hurts most. The same argument applies to biases and normalisation parameters, and it is the standard practice from the pruning literature: leave the first and last layers dense.

8. Component 4 - Mask-Aware Matmul

Source: ODT_Complete_Reference_v2.pdf, section 8.4 (primary) and section 39.5 (variant)

Purpose. Skip the multiply-accumulate for inactive weights. This is the component that converts sparsity into speed: at 90% sparsity, nine of every ten inner-loop iterations exit early. **Depends on:** Component 3.

```

1  /* Matmul.c - inside matmulFloatCore(), innermost loop */
2  for (size_t i = 0; i < aColumns; i++) {
3
4      size_t flatIdx = rowIndex * aColumns + i;
5
6      if (weightMask != NULL &&
7          !tensorBoolGet(weightMask, flatIdx))
8          continue;          /* inactive: contributes nothing */
9
10     float aVal = readBytesAsFloat(&A->data[aByteIdx]);
11     float bVal = readBytesAsFloat(&B->data[bByteIdx]);
12     result += aVal * bVal;
13 }

```

Listing 6.1 - The masked inner loop, from section 8.4.

8.1 Line by line

Line	Code	What it does and why
1	/* Matmul.c - inside matmulFloatCore() */	Does: locates the change. Why this function: ODT routes every float matrix multiply through this one kernel, so a single edit covers the forward pass of every linear layer in every model.
2	for (size_t i = 0; i < aColumns; i++) {	Does: the reduction loop - for one output element, sum over the shared dimension. aColumns is the number of input features, 1,152 for the HAR layer and 4 for the toy layer. Why this loop and not an outer one: it executes outputs x inputs = 73,728 times per sample, which makes it the hottest code in the library and the only place where skipping work is worth a branch.
4	size_t flatIdx = rowIndex * aColumns + i;	Does: converts the 2-D position (output neuron, input feature) into the 1-D index the mask uses. Toy layer, output 1 and input 2: $1 \times 4 + 2 = 6$. Why it is correct here: weights are stored [out, in] row-major and transposed in O(1) before the matmul, so rowIndex is the output neuron and i the input. Why it is fragile: that O(1) transpose is an assumption about the caller. If anyone ever makes it a real data movement, or changes the storage order, this line silently addresses the wrong bit. If wrong: no crash. You skip the wrong weights and the model trains to a plausible but incorrect result - the worst failure mode in this document. Chapter 3 gives the five-minute test that catches it.
6	if (weightMask != NULL &&	Does: tests whether this layer is sparse at all. Why NULL-first: C guarantees left-to-right short-circuit evaluation, so tensorBoolGet is never reached on a dense layer. This one test is what lets a single kernel serve both cases. Cost on dense layers: one perfectly predicted branch per iteration - a fraction of a cycle on an M7 with branch prediction, and the price of not maintaining two copies of the kernel.
7	!tensorBoolGet(weightMask, flatIdx))	Does: reads one mask bit; the negation selects INACTIVE weights. What it compiles to: <code>(data[idx>>3] >> (idx&7)) & 1</code> - a byte load, a shift and an AND, roughly 3-4 cycles. The break-even calculation: it is protecting one fused multiply-add, about 1 cycle. So the check pays only when it skips often. At 90% sparsity you spend 4 cycles to save 9×1 - a clear win. At 20% sparsity you spend 4 to save 0.25, and the masked kernel is SLOWER than the dense one. Sparse is not automatically fast.

Line	Code	What it does and why
8	<code>continue;</code>	Does: skips to the next input feature. Why nothing else happens here: no zero is accumulated, no counter incremented. The inactive weight is simply absent from the sum, which is arithmetically identical to multiplying by a stored zero - and that equivalence is exactly what the verification test in Chapter 14 exploits.
10	<code>float aVal = readBytesAsFloat(&A->data[aByteIdx]);</code>	Does: reads one activation. Why not a direct <code>*(float*)</code> dereference: tensor buffers are not guaranteed 4-byte aligned. On a Cortex-M7 an unaligned word load is legal but slower; on stricter cores it faults. <code>readBytesAsFloat</code> does the safe thing.
11	<code>float bVal = readBytesAsFloat(&B->data[bByteIdx]);</code>	Does: reads the corresponding weight. Note: this is reached only for active weights, so the memory traffic of the weight tensor also falls by the sparsity fraction - which on a bandwidth-bound device matters as much as the arithmetic saved.
12	<code>result += aVal * bVal;</code>	Does: the multiply-accumulate. Why it is one line and not two: an M7 with the single-precision FPU issues a fused multiply-add in one instruction, so the compiler should emit VFMA here. If it does not, check that <code>-ffp-contract=fast</code> is enabled.
13	<code>}</code>	Does: closes the reduction loop. What has changed overall: at 90% sparsity, nine of every ten iterations reach line 8 and stop. The instruction counter should report about 7,373 multiply-accumulates instead of 73,728 - the measurement that proves this edit is live.

Table 8.1 - Every line of the masked inner loop.

8.2 Plumbing the mask into the kernel

The listing assumes `weightMask` is in scope inside `matmulFloatCore()`, and in the existing code it is not. The source offers two options and recommends one:

Option	How	Verdict
Thread the parameter	Add a <code>tensor_t *weightMask</code> argument to <code>matmulFloatCore()</code> and a new public <code>matmulFloat32TensorsWithMask()</code> that passes it; existing callers pass NULL	Recommended. Explicit, re-entrant, and the compiler finds every call site for you
Global context variable	Set a file-scope mask pointer before calling the existing function	Rejected. Not re-entrant, breaks under any future threading, and a forgotten reset silently applies one layer's mask to another

8.3 Verifying it actually works

THE INSTRUCTION COUNTER TEST

ODT exposes `getMatmulInstructionCounter()`. Run one forward pass dense and one masked at 90% sparsity on the same 64 x 1152 layer. Dense should report 73,728 multiply-accumulates; masked should report roughly 7,373. If masked still reports 73,728, the mask is not reaching the kernel - most likely you added the parameter but a wrapper is still passing NULL. If it reports something between the two, your flat index is misaligned with the mask and you are skipping the wrong weights, which is far worse than not skipping at all because the model will still appear to train.

8.4 The gradient matmul must NOT be masked

THE SINGLE MOST IMPORTANT WARNING IN THIS DOCUMENT

RigL grows connections using the gradients of INACTIVE weights. Those gradients come from the weight-gradient matmul, $dL/dW = \delta^T x$, which is a different call from the forward matmul. If you apply the mask to the weight-gradient computation as well - which looks consistent and saves more compute - then every inactive weight has a gradient of exactly zero, `findAbsKthLargestInactive()` returns 0, and the GROW step selects essentially arbitrary positions. The model will still train, the sparsity will still look correct, and the mask will be evolving at random: you will have implemented static sparse training with extra steps. MASK THE FORWARD MATMUL AND THE LOSS-PROPAGATION MATMUL; LEAVE THE WEIGHT-GRADIENT MATMUL DENSE. Chapter 17, defect D3.

This is also where RigL's real cost sits. The forward pass gets nine tenths cheaper, but the weight-gradient pass stays dense, so the end-to-end training speedup is well under the 10x that the sparsity figure suggests. The published algorithm accepts exactly this trade: dense gradients are the price of learning the connectivity.

9. Component 5 - Mask-Aware SGD

Source: ODT_Complete_Reference_v2.pdf, section 12.4 (primary) and section 39.6 (variant)

Purpose. Stop the optimiser from moving inactive weights away from zero. Without this, weight decay, momentum and any residual gradient will gradually give every 'pruned' weight a non-zero value, and the sparsity becomes a fiction that only the mask believes in. **Depends on:** Component 3.

```

1  static void sgdUpdateKernelMasked(tensor_t **op, size_t n,
2                                     tensor_t *rawOut, tensor_t *aux,
3                                     const void *ctxv) {
4      const sgdUpdateCtx_t *ctx = ctxv;
5      tensor_t *mask = ctx->weightMask;      /* NULL for dense layers */
6
7      float *param = (float *)op[0]->data;
8      float *grad  = (float *)op[1]->data;
9      float *out   = (float *)rawOut->data;
10     size_t nElem = calcNumberOfElementsByTensor(rawOut);
11
12     for (size_t i = 0; i < nElem; i++) {
13
14         if (mask != NULL && !tensorBoolGet(mask, i)) {
15             out[i] = 0.0f;
16             grad[i] = 0.0f;
17             continue;
18         }
19
20         float g = grad[i] + ctx->weightDecay * param[i];
21         out[i] = param[i] - ctx->lr * g;
22     }
23 }
```

Listing 7.1 - Mask-aware SGD update, merged from sections 12.4 and 39.6.

9.1 Line by line

Line	Code	What it does and why
1-3	static void sgdUpdateKernelMasked(tensor_t **op, ...)	Does: declares an ODT kernel - invoked through the executeOp funnel with an operand array rather than named tensors. Why `static`: file-local; the funnel holds the pointer, so no external linkage is needed. Design note: the source presents this as a NEW function beside the existing <code>sgdUpdateKernel</code> . Adding the mask check to the existing one, with NULL meaning dense, would avoid two nearly identical kernels drifting apart.
4	const sgdUpdateCtx_t *ctx = ctxv;	Does: recovers the typed context from the funnel's <code>void *</code> . Why `const`: the kernel reads hyperparameters and must not mutate them; the qualifier lets the compiler keep <code>lr</code> in a register across the loop.
5	tensor_t *mask = ctx->weightMask;	Does: fetches the mask from the optimiser context. Why from the context and not the operands: operands are the tensors being updated; the mask is configuration. The plumbing step everyone forgets: <code>sgdUpdateCtx_t</code> needs this new field AND something must copy <code>linearConfig_t.weightMask</code> into it when the optimiser is configured. Miss that and the code compiles, runs, and masks nothing.
7	float *param = (float)op[0]->data;	Does: the current weights. Why index 0: the funnel's operand ordering convention - parameters first, gradients second.
8	float *grad = (float)op[1]->data;	Does: the gradients accumulated since the last update.

Line	Code	What it does and why
9	<code>float *out = (float *)rawOut->data;</code>	Does: where the new weights are written. Why a separate pointer: <code>out</code> and <code>param</code> MAY alias the same buffer for an in-place update. Nothing in this kernel breaks if they do, because every element is read before it is written.
10	<code>size_t nElem = calcNumberOfElementsByTensor(rawOut);</code>	Does: the element count, taken from the OUTPUT tensor. If wrong: the mask must have exactly this many bits. A mismatch reads past the mask's storage - one of the few ways this code can corrupt memory rather than merely compute the wrong answer.
12	<code>for (size_t i = 0; i < nElem; i++) {</code>	Does: one pass over every parameter. Cost context: this runs once per optimiser step, not once per multiply-accumulate, so unlike the <code>matmul</code> the branch below costs nothing worth measuring. The mask check here is about correctness.
14	<code>if (mask != NULL && !tensorBoolGet(mask, i)) {</code>	Does: selects inactive weights. Why the same NULL-first pattern as the <code>matmul</code>: dense layers must behave exactly as before, so that adding RigL cannot regress any existing model.
15	<code>out[i] = 0.0f;</code>	Does: forces the weight to exactly zero. Why not simply skip the write: if you <code>continue</code> without writing, the weight keeps its old value. The masked <code>matmul</code> ignores it, so every test still passes - until the model is checkpointed and reloaded with a lost or misaligned mask, at which point untrained values are suddenly live. Writing the zero puts the invariant in the DATA, and the data is what survives serialization.
16	<code>grad[i] = 0.0f;</code>	Does: clears the gradient accumulator for this position. Why gradients need clearing at all: ODT accumulates gradients across micro-batches, so without a reset the sums grow without bound. The tension with RigL: this line destroys exactly the signal GROW ranks by. It is safe only because <code>rigLStep()</code> runs BEFORE the optimiser in the same iteration. Reverse the order and every inactive gradient is zero by the time GROW looks - defects D3 and D4.
17	<code>continue;</code>	Does: skips the update arithmetic for this element.
18	<code>}</code>	Does: closes the inactive branch. Missing here: the momentum buffer. If your SGD carries velocity, a regrown weight inherits the velocity it had before it was dropped and takes its first step in a stale direction. Clear it in this branch. Defect D5.
20	<code>float g = grad[i] + ctx->weightDecay * param[i];</code>	Does: adds L2 weight decay to the gradient. Why COUPLED and not decoupled: for plain SGD the two formulations differ only by a factor of the learning rate, so folding decay into the gradient is standard. AdamW is the case where the distinction matters, because its adaptive scaling would otherwise divide the decay away - see Chapter 10, line 15.
21	<code>out[i] = param[i] - ctx->lr * g;</code>	Does: the gradient descent step. Why this is the whole of SGD: everything else in this kernel is bookkeeping. Momentum, if configured, is applied elsewhere in the ODT pipeline, which is why it is not visible - and why it is easy to forget when adding the mask.
22-23	<code>} }</code>	Does: closes the loop and the function. Postcondition worth asserting in a test: after this kernel, every position whose mask bit is 0 holds exactly 0.0f in both <code>out</code> and <code>grad</code> . That is a two-line check and it catches most plumbing mistakes.

Table 9.1 - Every line of the mask-aware SGD kernel.

WHY FORCING THE WEIGHT TO ZERO MATTERS MORE THAN IT LOOKS

Suppose you skip line 15 and simply `continue` without writing. The inactive weight keeps whatever value it had. The masked `matmul` still ignores it, so the forward pass is unchanged and every test passes. Then you checkpoint the model, and on reload the mask is gone or misaligned - and suddenly those weights are live again with values that were never trained. Writing the zero makes the invariant 'inactive implies zero' hold in the DATA, not just in the mask, and the data is what survives serialization.

10. Component 5b - Mask-Aware AdamW

Source: ODT_Complete_Reference_v2.pdf, section 13.4

Purpose. The same protection as Component 5, plus one extra step that SGD does not need: clearing the first and second moment buffers for inactive weights. **Depends on:** Component 3. **Optional** - needed only if you train with AdamW rather than SGD.

```

1  for (size_t i = 0; i < n; i++) {
2
3      if (mask != NULL && !tensorBoolGet(mask, i)) {
4          param[i] = 0.0f;
5          grad[i] = 0.0f;
6          m[i] = 0.0f;      /* clear first-moment history */
7          v[i] = 0.0f;      /* clear second-moment history */
8          continue;
9      }
10
11     float g = grad[i];
12     m[i] = beta1 * m[i] + (1.0f - beta1) * g;
13     v[i] = beta2 * v[i] + (1.0f - beta2) * g * g;
14     param[i] -= lrCorr * m[i] / (sqrtf(v[i]) + eps);
15     param[i] -= lr * weightDecay * param[i];
16     grad[i] = 0.0f;
17 }

```

Listing 8.1 - Mask-aware AdamW, from section 13.4.

10.1 Line by line - what differs from SGD

Line	Code	What it does and why
1	for (size_t i = 0; i < n; i++) {	Does: one pass over every parameter, exactly as in SGD. Why AdamW needs its own kernel at all: it carries two extra per-parameter state tensors, m and v , and those must be masked too - which is the whole reason this component exists separately.
3	if (mask != NULL && !tensorBoolGet(mask, i)) {	Does: selects inactive weights, same NULL-first pattern as everywhere else.
4	param[i] = 0.0f;	Does: forces the weight to exactly zero, as in SGD. Why it is written to `param` here and to `out` in the SGD kernel: the AdamW kernel updates in place, so there is no separate output tensor. Check which convention your ODT build uses before copying either listing.
5	grad[i] = 0.0f;	Does: clears the gradient accumulator. Same tension as SGD line 16: this destroys the signal GROW needs, and is safe only because rigLStep() runs first in the iteration.
6	m[i] = 0.0f;	Does: clears the first-moment estimate - the exponential moving average of the gradient. Why this line exists, concretely: suppose a weight is dropped at step 1000 and regrown at step 1500. Without clearing, m still holds the average gradient from before the drop. On the first step after regrowth, Adam applies that stale direction at full size to a weight that was just reset to zero - a large, confident step computed from history that no longer describes the loss surface. Clearing it means the regrown weight's first update comes only from its current gradient, exactly as a freshly initialised parameter's would.
7	v[i] = 0.0f;	Does: clears the second-moment estimate - the moving average of the squared gradient. Why the effect is subtler than for `m`: Adam divides by $\sqrt{v_hat}$, so a stale LARGE v would make the regrown weight almost frozen - it would sit at zero and contribute nothing for hundreds of steps, quietly wasting the connection RigL just spent budget to create. With v cleared, the bias correction makes the first step approximately +/- the learning rate, the same size any fresh parameter takes.

Line	Code	What it does and why
8	<code>continue;</code>	Does: skips the Adam arithmetic. Also missing here, as in SGD: nothing resets the step counter used for bias correction. Adam's correction terms depend on a global <code>t</code> , so a regrown weight is treated as if it had been training since step 0. With <code>m</code> and <code>v</code> both zero the practical effect is small, but it is worth knowing when reading the first update after a grow.
9	<code>}</code>	Does: closes the inactive branch. Everything below is reached only by active weights.
11	<code>float g = grad[i];</code>	Does: takes the raw gradient. Note what is NOT added here: weight decay. Unlike the SGD kernel, which folds decay into <code>g</code> , AdamW applies it separately on line 15 - that separation is the entire difference between Adam and AdamW.
12	<code>m[i] = beta1 * m[i] + (1.0f - beta1) * g;</code>	Does: updates the first moment - an exponential moving average of the gradient with <code>beta1</code> , typically 0.9. Intuition: momentum. It smooths the noisy per-batch gradient into a running direction, so that consistent signal accumulates and oscillation cancels.
13	<code>v[i] = beta2 * v[i] + (1.0f - beta2) * g * g;</code>	Does: updates the second moment - the moving average of the SQUARED gradient, with <code>beta2</code> typically 0.999. Intuition: a per-parameter estimate of how large gradients usually are here. Dividing by its square root on the next line gives every parameter a step size scaled to its own gradient history, which is why Adam needs so little tuning across layers of very different magnitudes.
14	<code>param[i] -= lrCorr * m[i] / (sqrtf(v[i]) + eps);</code>	Does: the Adam step. Why <code>lrCorr</code> rather than <code>lr</code>: it folds in both bias corrections, $1/(1-\text{beta1}^t)$ and $1/(1-\text{beta2}^t)$, so the division happens once per element instead of twice - worth doing on an M7 where a float divide costs about 14 cycles. Why <code>+ eps</code> and not a zero check: <code>v</code> is zero for a parameter that has never seen a gradient - including every weight RigL just grew - so without <code>eps</code> this divides by zero on the first step after every single grow.
15	<code>param[i] -= lr * weightDecay * param[i];</code>	Does: decoupled weight decay, applied to the parameter directly. Why it is a separate line and not folded into <code>g</code>: if decay went through the gradient it would be divided by $\sqrt{v}$ along with everything else, so parameters with large gradients would be barely regularised and parameters with small ones heavily. Keeping it outside the adaptive scaling is the <code>W</code> in AdamW, and it reliably improves generalisation.
16	<code>grad[i] = 0.0f;</code>	Does: resets the accumulator for the next micro-batch, on the ACTIVE path this time.
17	<code>}</code>	Does: closes the loop. Memory reality check: this kernel touches four dense float tensors per layer - <code>param</code> , <code>grad</code> , <code>m</code> , <code>v</code> - which for the HAR 1152 x 64 layer is 1.13 MB. That does not fit in 320 KB, whatever the sparsity, because <code>m</code> and <code>v</code> are allocated densely regardless of the mask. Defect D6.

Table 10.1 - Every line of the mask-aware AdamW update.

ALSO CLEAR THE SGD MOMENTUM BUFFER

Section 13.4 correctly identifies stale moments as a hazard for AdamW and says nothing about SGD with momentum, which has exactly the same problem in a milder form: a regrown weight inherits the velocity it had before it was dropped. If your SGD carries a momentum buffer, clear that entry too, in the branch on line 14-18 of Listing 7.1. Chapter 17, defect D5.

10.2 Memory cost of AdamW under sparsity

Tensor	Elements	Bytes	Note
Weights	73,728	294,912	Dense storage regardless of mask
Gradients	73,728	294,912	Must stay dense - GROW needs them
m (first moment)	73,728	294,912	Allocated densely

Tensor	Elements	Bytes	Note
v (second moment)	73,728	294,912	Allocated densely
Mask	73,728 bits	9,216	3% overhead
Total for one layer	-	1,188,864	1.13 MB - exceeds the 320 KB SRAM on its own

ADAMW PLUS THIS LAYER DOES NOT FIT ON THE TARGET

Section 13.4 states the AdamW cost as 88 KB by counting only the 7,373 ACTIVE weights. That figure is right only if the moment tensors are stored sparsely - and nothing in the implementation does that; **m** and **v** are allocated with the same dense shape as the parameters. The honest number for this layer is 1.13 MB, which does not fit. Either use SGD (which needs no moment buffers and brings the layer to 599 KB - still too large), or shrink the layer, which is what the source itself recommends in section 39.10 when it suggests a Conv1d feature extractor ahead of a much smaller linear head. Chapter 17, defect D6.

11. Component 6 - rigLStep(), the Algorithm Itself

Source: ODT_Complete_Reference_v2.pdf, section 39.7, with the DROP/GROW logic cross-checked against sections 2.2 and 33.4

Purpose. Everything so far has been scaffolding. This function is RigL: it computes the swap size from the cosine schedule, drops the weakest active weights, grows the most promising inactive ones, and leaves the layer with the same number of active weights it started with. **Depends on:** Components 1, 2 and 3.

```

1 void rigLStep(layer_t **model, size_t numLayers,
2               float sparsityTarget, size_t step, size_t totalSteps) {
3
4     /* Cosine decay of the swap fraction. */
5     float prog = (float)step / (float)(totalSteps > 0 ? totalSteps : 1);
6     float alpha = 0.3f * 0.5f * (1.0f + cosf(3.14159265f * prog));
7
8     for (size_t l = 0; l < numLayers; l++) {
9
10        if (model[l]->type != LINEAR) continue;
11
12        linearConfig_t *cfg = model[l]->config->linear;
13        tensor_t *mask = cfg->weightMask;
14        if (mask == NULL) continue;          /* dense layer */
15
16        tensor_t *weights = cfg->weights->param;
17        tensor_t *grads = cfg->weights->grad;
18        size_t n = calcNumberOfElementsByTensor(weights);
19        float *w = (float *)weights->data;
20        float *g = (float *)grads->data;
21
22        size_t numActive = 0;
23        for (size_t i = 0; i < n; i++)
24            if (tensorBoolGet(mask, i)) numActive++;
25
26        size_t K = (size_t)(alpha * (float)numActive);
27        if (K == 0) continue;                /* nothing to swap */
28
29        /* ---- DROP: deactivate the K smallest |w| ---- */
30        float dropThresh = findAbsKthSmallestActive(weights, mask, K);
31        for (size_t i = 0; i < n; i++) {
32            if (tensorBoolGet(mask, i) && fabsf(w[i]) <= dropThresh) {
33                tensorBoolSet(mask, i, false);
34                w[i] = 0.0f;
35            }
36        }
37
38        /* ---- GROW: activate the K largest |g| among inactive ---- */
39        float growThresh = findAbsKthLargestInactive(grads, mask, K);
40        for (size_t i = 0; i < n; i++) {
41            if (!tensorBoolGet(mask, i) && fabsf(g[i]) >= growThresh) {
42                tensorBoolSet(mask, i, true);
43                w[i] = 0.0f;                /* grow at zero */
44            }
45        }
46
47        /* ---- Zero the gradients of still-inactive weights ---- */
48        for (size_t i = 0; i < n; i++)
49            if (!tensorBoolGet(mask, i)) g[i] = 0.0f;
50    }
51 }

```

Listing 9.1 - rigLStep(), transcribed from section 39.7.

11.1 Line by line

Line	Code	What it does and why
1	<code>void rigLStep(layer_t **model,</code>	Does: declares the entry point. Returns <code>void</code> - the function reports nothing and mutates the model in place. Why: <code>layer_t **</code> is an array of layer pointers, the same representation the training loop already holds, so no conversion is needed at the call site. If wrong: passing a <code>layer_t *</code> (array of structs) instead of <code>layer_t **</code> compiles with a warning and then walks memory at the wrong stride.
2	<code>size_t numLayers, float sparsityTarget,</code>	Does: the layer count and, nominally, the target sparsity. Why: <code>numLayers</code> is needed because C arrays carry no length. Note: <code>sparsityTarget</code> is never read in the body . Sparsity is fixed once by <code>bernoulliFillMask()</code> at initialisation; this function only preserves whatever the mask already encodes. The parameter is misleading and should be removed or honoured.
2	<code>size_t step, size_t totalSteps) {</code>	Does: the current global step and the schedule horizon. Why: the pair defines training progress, which drives alpha. If wrong: passing the true total instead of <code>0.8 x total</code> leaves alpha non-zero into the final epoch, so connections are still being swapped when there is no time left to train them - defect D9.
4	<code>/* Cosine decay of the swap fraction. */</code>	Does: nothing at runtime. Why: marks the boundary between the schedule computation, which is per-call, and the per-layer loop that follows.
5	<code>float prog = (float)step / (float)(totalSteps > 0 ? ... : 1);</code>	Does: training progress as a fraction in <code>[0, 1]</code> . Why: both operands are cast to <code>float</code> FIRST. <code>step/totalSteps</code> in integer arithmetic would be 0 for the entire run except the last step, making alpha constant at <code>alphaInit</code> and the mask thrash until training ended. The ternary guards a caller that passes 0. If wrong: dropping either cast is the single most likely silent bug in this function - it compiles, runs, and produces a schedule that never decays.
6	<code>float alpha = 0.3f * 0.5f * (1.0f + cosf(3.14159265f * prog));</code>	Does: evaluates $\alpha(t) = (\alpha_{init}/2)(1 + \cos(\pi t / T_{end}))$. At <code>prog=0</code> , $\cos(0)=1$ so $\alpha=0.3$; at <code>prog=1</code> , $\cos(\pi)=-1$ so $\alpha=0$. Why the halving: $\cos$ ranges over <code>[-1, 1]</code> , so $(1+\cos)$ ranges over <code>[0, 2]</code> ; multiplying by 0.5 brings it to <code>[0, 1]</code> and by 0.3 to <code>[0, 0.3]</code> . Why cosine and not linear: the derivative is zero at both ends, so the swap rate changes slowly at the start (while the network is still finding structure) and slowly at the end (while it settles), and fastest in the middle. Note: 0.3f and the constant π are hard-coded; both belong in parameters.
8	<code>for (size_t l = 0; l < numLayers; l++) {</code>	Does: iterates every layer of the model. Why: one call updates the whole model, so the training loop needs a single line rather than a loop of its own. Note that alpha is computed once, outside this loop - every layer swaps the same <code>FRACTION</code> , not the same <code>COUNT</code> .
10	<code>if (model[l]->type != LINEAR) continue;</code>	Does: skips anything that is not a fully connected layer. Why this is not optional: <code>config</code> is a UNION. Reading <code>->linear</code> on a <code>Conv1d</code> layer does not fail - it reinterprets those bytes as a <code>linearConfig_t</code> and hands you a garbage <code>weightMask</code> pointer that will be dereferenced two lines later. Consequence: conv kernels are never sparsified, even though section 20.4 of the source describes a <code>kernelMask</code> for them.
12	<code>linearConfig_t *cfg = model[l]->config->linear;</code>	Does: narrows the union to the linear view. Why: every field this function needs - the mask, the weights, the gradients - hangs off this one pointer, so it is worth naming once rather than repeating the chain.
13	<code>tensor_t *mask = cfg->weightMask;</code>	Does: fetches the layer's mask. Why: this single field is what distinguishes a sparse layer from a dense one; everything below branches on it.
14	<code>if (mask == NULL) continue;</code>	Does: skips dense layers. Why: combined with line 10, this makes <code>rigLStep()</code> safe to call on any model at all - including a fully dense one, where it does nothing. That is what lets the training loop call it unconditionally. If wrong: omitting it dereferences <code>NULL</code> on the first dense layer, which at least fails loudly.

Line	Code	What it does and why
16	<code>tensor_t *weights = cfg->weights->param;</code>	Does: the weight tensor. Why: DROP ranks by <code> weight </code> , so this is the DROP input. Note <code>cfg->weights</code> is a <code>parameter_t</code> , which bundles the values and their gradients - line 17 takes the other half.
17	<code>tensor_t *grads = cfg->weights->grad;</code>	Does: the gradient tensor, filled by the backward pass of this same iteration. Why: GROW ranks by <code> gradient </code> . Critical: these gradients must be present for INACTIVE weights, which means the weight-gradient matmul must not be masked and the optimiser must not have run yet. This one line is where defect D3 bites.
18	<code>size_t n = calcNumberOfElements ByTensor(weights);</code>	Does: the element count - 73,728 for the HAR layer, 8 for the toy layer of Chapter 2. Why: every loop below runs over it, and the mask must have exactly this many bits. If wrong: a mask sized differently from the weights reads past its own buffer.
19	<code>float *w = (float)weights->data;</code>	Does: a raw float view of the weights. Why: the tensor accessors would cost a function call per element inside three $O(n)$ loops. If wrong: valid ONLY for FLOAT32. Under FQT the tensor holds int32 mantissas, and this cast reinterprets their bit patterns as floats - producing magnitudes with no relation to the real weights, so DROP selects essentially at random.
20	<code>float *g = (float)grads->data;</code>	Does: the same for gradients. Why and if wrong: identical to line 19; in ODT the gradient tensor is if anything MORE likely to be SYM_INT32 than the weight tensor.
22	<code>size_t numActive = 0;</code>	Does: initialises the active-weight counter. Why: the count is not stored anywhere - the mask is the only record of it, so it must be recomputed each time.
23-24	<code>for (i...) if (tensorBoolGet(mask, i)) numActive++;</code>	Does: counts set bits. For the toy layer: 4. For the HAR layer at initialisation: about 7,412. Why: K is a fraction of the ACTIVE count, not of n. Cost: this is the third full pass over the mask in this function and <code>findAbsKthSmallestActive()</code> will make two more; caching the count in the layer config would remove three $O(n)$ scans per step.
26	<code>size_t K = (size_t)(alpha * (float)numActive);</code>	Does: how many weights to swap. Toy layer: $0.5 \times 4 = 2$. HAR at step 0: $0.3 \times 7,412 = 2,223$. Why the cast: truncation toward zero implements the floor() of the published algorithm. Why active and not n: section 2.2 defines $K = \alpha (1-s) N$, and numActive IS $(1-s)N$ - so this matches, and it also means K shrinks automatically if sparsity ever drifts.
27	<code>if (K == 0) continue;</code>	Does: skips the layer once the swap size rounds to zero. Why: this is how the mask freezes at the end of the schedule - there is no separate end-of-schedule test. It also protects <code>findAbsKthSmallestActive(..., 0)</code> from being asked for a zeroth-smallest element.
29	<code>/* ---- DROP: deactivate the K smallest w ---- */</code>	Does: nothing at runtime, but marks the first of the three phases - DROP, GROW, clear - that every reader should be able to name.
30	<code>float dropThresh = findAbsKthSmallestActive(weights, mask, K);</code>	Does: computes the DROP threshold using Component 1. Toy layer: 0.30. Why it is computed BEFORE the loop: the threshold must describe the mask as it was at the start of the step. Recomputing it inside the loop would make each decision depend on the previous ones.
31	<code>for (size_t i = 0; i < n; i++) {</code>	Does: walks every weight, active or not. Why not just the active ones: there is no list of active indices - the mask is a bitmap, so finding them requires this scan anyway.
32	<code>if (tensorBoolGet(mask, i) && fabsf(w[i]) <= dropThresh) {</code>	Does: selects active weights at or below the threshold. Why the mask test first: short-circuit evaluation skips the <code>fabsf</code> for the 90% of weights that are inactive. If wrong - and it is wrong as written: the non-strict <code><=</code> combined with a zero-indexed K-th value selects K+1 weights, and more when magnitudes tie. In the toy layer it drops index 4 ($ w = 0.30 =$ the threshold) which should have survived. Use a strict <code><</code> . Defect D1.
33	<code>tensorBoolSet(mask, i, false);</code>	Does: clears the mask bit - the connection is now inactive. Why here rather than in a second pass: the GROW loop that follows reads this same mask, so the DROP must be complete first.

Line	Code	What it does and why
34	<code>w[i] = 0.0f;</code>	Does: zeroes the weight itself. Why it matters more than it looks: without it the value survives in the tensor, invisible because the matmul skips it - until the model is serialized and reloaded with a lost or misaligned mask, at which point untrained values come back to life. Writing the zero puts the invariant in the DATA, and the data is what persists.
35-36	<code>} }</code>	Does: closes the conditional and the DROP loop. At this point the layer is temporarily BELOW its target density - the toy layer has 2 active weights instead of 4.
38	<code>/* ---- GROW: activate K largest g among inactive ---- */</code>	Does: marks phase two. Why the order matters: DROP frees the budget that GROW spends. Reversing them would push the layer above target density before trimming it back.
39	<code>float growThresh = findAbsKthLa rgestInactive(grads, mask, K);</code>	Does: computes the GROW threshold using Component 2. Toy layer: 0.40. If wrong - and it is: <code>mask</code> here is the POST-drop mask, so the K weights just deactivated are candidates for immediate reactivation. In the toy layer index 1 is dropped for $ w = 0.05$ and instantly regrown for $ g = 0.85$, destroying a trained value and consuming part of the swap budget. Pass a snapshot of the pre-drop mask. Defect D2.
40	<code>for (size_t i = 0; i < n; i++) {</code>	Does: a second full scan. Why a separate loop: the threshold on line 39 must see the completed DROP, so the two phases cannot be fused into one pass.
41	<code>if (!tensorBoolGet(mask, i) && fabsf(g[i]) >= growThresh) {</code>	Does: selects inactive weights whose gradient is at or above the threshold. Why gradient and not weight: an inactive weight is zero, so its magnitude carries no information; its GRADIENT says how much the loss would fall if it were switched on. This is the entire idea of RigL in one condition. If wrong: the non-strict <code>>=</code> mirrors defect D1 and grows K+1. The two errors cancel in the count, which is luck rather than design.
42	<code>tensorBoolSet(mask, i, true);</code>	Does: sets the mask bit - the connection is live from the next forward pass onward.
43	<code>w[i] = 0.0f; /* grow at zero */</code>	Does: starts the new connection at exactly zero. Why not at a random value: the network is mid-convergence; injecting an arbitrary weight is a perturbation with no justification. Starting at zero means the connection contributes nothing at first and is shaped entirely by gradients - it earns its value. Consequence: a grown weight needs time to become useful, which is exactly why the mask must freeze well before training ends.
44-45	<code>} }</code>	Does: closes the GROW loop. Density is now back at target - the toy layer has 4 active weights again, at positions {0, 4, 5, 7}.
47	<code>/* ---- Zero gradients of still-inactive weights ---- */</code>	Does: marks phase three, the one most readers skip.
48-49	<code>for (i...) if (!tensorBoolGet(mask, i)) g[i] = 0.0f;</code>	Does: clears the gradient of every weight that remains inactive. Why: without it, a weight that once had a large gradient keeps that stale value and looks permanently attractive to every future GROW step, so the same few positions would be selected again and again. Interaction: these gradients will be refilled by the next backward pass, which is why the dense weight-gradient matmul is non-negotiable.
50	<code>}</code>	Does: closes the per-layer loop. Each layer is handled independently and shares only the alpha computed on line 6.
51	<code>}</code>	Does: end of function. Nothing is returned and nothing is allocated, so there is nothing for the caller to clean up - which is why the D2 fix in Appendix A has to be careful to free the mask snapshot it introduces.

Table 11.1 - Every line of `rigLStep()`. Values quoted as 'toy layer' refer to the 8-weight example computed in Chapter 2.

11.2 What one call actually does, in order

```

step 900, alpha = 0.28, layer has 7,373 active of 73,728

1. K = floor(0.28 * 7373) = 2,064
2. dropThresh = 2064-th smallest |w| among the 7,373 active
3. DROP : ~2,065 weights mask 1->0, w := 0
      active: 7,373 -> 5,308
4. growThresh = 2064-th largest |g| among the now-68,420 inactive
5. GROW : ~2,065 weights mask 0->1, w := 0
      active: 5,308 -> 7,373      <- conserved
6. zero g[i] for all 66,355 still-inactive weights

net effect: 2,065 connections have moved to better positions,
            the active count is unchanged, and 2,065 weights
            that were carrying trained values are now zero.

```

Figure 9.1 - One rigLStep() on the HAR layer at step 900.

THE COST NOBODY MENTIONS

Every swap throws away a trained weight and installs a zero. At $\alpha = 0.3$ that is 28% of the layer's learned values discarded in a single step. This is why the cosine decay is not optional and why the interval T matters: swap too often, or keep α high for too long, and the network spends the whole run relearning connections instead of converging. If your loss spikes at every multiple of 100 steps and does not recover before the next one, α is too high or T is too small.

12. Component 7 - Persisting the Mask

Source: ODT_Complete_Reference_v2.pdf, section 17.5 (both functions) and section 39.8 (write only)

Purpose. Save the mask with the checkpoint and load it back. After 50 epochs the mask IS the learned architecture - which 10% of connections matter - and a checkpoint without it restores a model that is dense, wrong, and 10x slower. **Depends on:** Component 3.

```

1 void serializeSparsity(tensor_t *mask, FILE *fp) {
2
3     if (mask == NULL || mask->quantization->type != BOOL) {
4         uint8_t noMask = 0;
5         fwrite(&noMask, 1, 1, fp);
6         return;
7     }
8
9     uint8_t hasMask = 1;
10    fwrite(&hasMask, 1, 1, fp);
11
12    size_t n      = calcNumberOfElementsByTensor(mask);
13    size_t bytes = (n + 7) / 8;
14    fwrite(mask->data, 1, bytes, fp);
15 }
16
17 tensor_t *deserializeSparsity(size_t n, FILE *fp) {
18
19     uint8_t hasMask = 0;
20     fread(&hasMask, 1, 1, fp);
21     if (!hasMask) return NULL;          /* dense layer */
22
23     tensor_t *mask = allocBoolTensor(n);
24     size_t bytes = (n + 7) / 8;
25     fread(mask->data, 1, bytes, fp);
26     return mask;
27 }

```

Listing 10.1 - Mask serialization, from section 17.5.

12.1 Line by line

Line	Code	What it does and why
1	<code>void serializeSparsity(tensor_t *mask, FILE *fp) {</code>	Does: writes one mask to an open file. Why it takes the mask as a parameter: the original stub took no arguments, which is why it could not work. The mask lives on the layer config, so the caller - <code>serializeLinear()</code> - must pass it down. Why `FILE` and not a buffer: ODT checkpoints stream to disk or SD card; buffering a 294 KB tensor first would need memory the MCU does not have.
3	<code>if (mask == NULL mask->quantization->type != BOOL) {</code>	Does: two guards in one. NULL means a dense layer; a non-BOOL tensor means something else was attached by mistake. Why the second check matters: without it, attaching a FLOAT32 tensor by accident would write <code>(n+7)/8</code> bytes of float data as if it were a mask - a file that loads without error and produces a meaningless mask. Failing safe to 'no mask' is the right degradation.
4	<code>uint8_t noMask = 0;</code>	Does: the presence flag, cleared. Why `uint8_t` and not `int`: the file format is byte-exact and must not depend on the host's <code>int</code> width or endianness.

Line	Code	What it does and why
5	<code>fwrite(&noMask, 1, 1, fp);</code>	Does: writes one zero byte. Why write anything at all for a dense layer: so the reader always consumes exactly one byte here and learns from its value whether more follows. A format where a field is sometimes absent cannot be parsed without out-of-band knowledge. Not checked: the return value. A full disk or a failed SD write silently truncates the checkpoint.
6	<code>return;</code>	Does: ends the dense case. The record is exactly one byte.
9	<code>uint8_t hasMask = 1;</code>	Does: the presence flag, set.
10	<code>fwrite(&hasMask, 1, 1, fp);</code>	Does: writes it. Why the flag precedes the data: a streaming reader never has to seek backwards - it reads one byte, then knows exactly how many follow.
12	<code>size_t n = calcNumberOfElementsByTensor(mask);</code>	Does: the number of BITS, since a BOOL tensor's element count is its bit count. Why n is not written to the file: it is implied by the weight tensor that precedes this record. That saves four bytes per layer and couples the two records - a trade the format makes deliberately, and the reason <code>deserializeSparsity()</code> takes n as an argument.
13	<code>size_t bytes = (n + 7) / 8;</code>	Does: ceiling division - how many whole bytes hold n bits. Why <code>(n+7)/8</code> and not <code>ceil(n/8.0)</code>: exact integer arithmetic, no floating point, no rounding surprises, and it is the standard C idiom. For 73,728 bits it gives exactly 9,216; for 73,730 it would give 9,217 with six bits of the last byte unused.
14	<code>fwrite(mask->data, 1, bytes, fp);</code>	Does: writes the packed bit array verbatim. What this bakes into the format: the in-memory bit order. A file written by the host trainer is readable on the MCU only if both pack bits identically. ODT defines <code>tensorBoolGet</code> as <code>(data[i>>3] >> (i&7)) & 1</code> on both, so they do - but that agreement is undocumented and deserves a comment here, because it is the kind of assumption that breaks silently when someone optimises the accessor.
15	<code>}</code>	Does: ends the write path. Total cost for the HAR layer: 9,217 bytes, against 294,912 for the weights - about 3%.
18	<code>tensor_t *deserializeSparsity(size_t n, FILE *fp) {</code>	Does: the reader. Why it returns a tensor rather than filling one: the caller does not know whether a mask exists until the flag is read, so allocation has to happen here. Consequence: the caller owns the result and must attach it to <code>cfg->weightMask</code> and eventually free it - the second place in this implementation where mask ownership is undefined.
20	<code>uint8_t hasMask = 0;</code>	Does: initialises the flag. Why initialising matters: if the <code>fread</code> on the next line fails, this stays 0 and the function returns NULL - degrading to a dense layer rather than acting on a garbage flag.
21	<code>fread(&hasMask, 1, 1, fp);</code>	Does: reads the presence byte. Not checked: the return value again. A truncated file yields a partly-initialised mask and no error, which on an embedded target reading from an SD card is not a theoretical risk.
22	<code>if (!hasMask) return NULL;</code>	Does: returns NULL for a dense layer. Why NULL is exactly right: it matches the default set in Component 3, so a dense checkpoint loads into a dense layer with no special case anywhere.
24	<code>tensor_t *mask = allocBoolTensor(n);</code>	Does: allocates a bit-packed tensor of n bits. Why n comes from the caller: as noted on line 12, the bit count is not in the file - the caller derives it from the weight tensor it just read.
25	<code>size_t bytes = (n + 7) / 8;</code>	Does: the same ceiling division as the writer. Why it must be the same expression: writer and reader compute the record length independently. If they ever disagree - say one uses <code>n/8</code> - every subsequent record in the file is misaligned, and the failure appears in a completely different layer.
26	<code>fread(mask->data, 1, bytes, fp);</code>	Does: reads the packed bits straight into the tensor's buffer.

Line	Code	What it does and why
27	<code>return mask;</code>	Does: hands ownership to the caller. What the caller must then do: assign it to <code>cfg->weightMask</code> , and - worth asserting - check that the restored active count matches what was saved. A mask that loads with the wrong bit order still yields a plausible sparsity while being a completely different network.

Table 12.1 - Every line of mask serialization and deserialization.

12.2 Wiring it into the tensor record

```

1 void serializeTensor(tensor_t *t, tensor_t *mask, FILE *f) {
2     serializeShape(t->shape, f);
3     serializeQuantization(t->quantization, f);
4     serialWriteBytes(t->data, calcBytesPerTensor(t), f);
5     serializeSparsity(mask, f);          /* now saves the mask */
6 }
7
8 /* called from serializeLinear() as: */
9 serializeTensor(weights, cfg->weightMask, f);

```

Listing 10.2 - Threading the mask through serializeTensor(), from section 17.5.

THIS CHANGES A FORMAT THE SOURCE CALLS 'LOCKED'

Chapter 17 of the source describes the checkpoint format as 'Locked Binary Format v2', and adding a presence byte to every tensor record changes it for DENSE models too - old files no longer parse, and old readers cannot read new files. If the format really is locked, this belongs behind a version bump to v3, with the reader accepting both. The change is one byte per tensor, so the cost of doing it properly is trivial; the cost of not doing it is a checkpoint corpus that silently splits in two. Chapter 17, defect D9.

What is stored	Bytes for the HAR 1152 x 64 layer
Weight data	294,912
Presence flag	1
Packed mask	9,216
Mask overhead	3.1% of the record

13. Components 8 to 10 - Initialization, Training Loop, Schedules

Source: ODT_Complete_Reference_v2.pdf, sections 9.4, 14.4, 37.4, 38.4 and 39.9

13.1 Component 8 - creating the mask (UserAPI)

Nothing built so far ever creates a mask. `buildSequentialModel()` must do it when a layer spec asks for sparsity.

```

1  /* UserAPI.c - inside buildSequentialModel(), per LINEAR layer spec */
2  if (spec->sparse) {
3      size_t maskBits = inFeat * outFeat;
4      tensor_t *mask = createBoolTensor(maskBits);
5      bernoulliFillMask(mask, 1.0f - spec->sparsity);
6      zeroInactiveWeights(linearCfg, mask);
7      linearCfg->weightMask = mask;
8  }
```

Listing 11.1 - Mask creation, from section 38.4.

Line	Code	What it does and why
2	<code>if (spec->sparse)</code>	A new field on <code>layerSpec_t</code> , alongside a <code>float sparsity</code> . Layers that do not ask for sparsity are untouched and keep <code>weightMask = NULL</code> .
3	<code>maskBits = inFeat * outFeat</code>	One bit per weight. This must equal <code>calcNumberOfElementsByTensor(weights)</code> exactly - if the layer has a bias, it is NOT masked, and the bias is a separate tensor, so the product of the two feature counts is right.
4	<code>createBoolTensor(maskBits)</code>	Allocates <code>ceil(bits/8)</code> bytes of bit-packed storage plus the tensor header.
5	<code>bernoulliFillMask(mask, 1.0f - spec->sparsity)</code>	The argument is P(active). For <code>spec->sparsity = 0.9</code> this passes 0.1. Because the draws are independent, the realised active count is binomial around 7,373 with a standard deviation of about 81 - so your first log line will read 7,412 active, not 7,373, and that is correct behaviour, not a bug.
6	<code>zeroInactiveWeights(linearCfg, mask)</code>	Enforces the invariant at initialisation: every inactive weight is set to exactly zero. Section 11.4 omits this call and section 38.4 includes it; without it, inactive positions keep their random init values, which then appear in the checkpoint and come back to life if the mask is ever lost.
7	<code>linearCfg->weightMask = mask;</code>	Attach. From this point the matmul, the optimiser and <code>rigLStep()</code> all see the layer as sparse.

SEED BEFORE YOU FILL

Section 9.4 makes a point worth repeating: call `rngSetSeed()` before `bernoulliFillMask()` AND before the training loop. The mask draw consumes RNG state that stochastic rounding would otherwise have used, so the two are coupled - the same seed gives you the same mask and the same rounding sequence only if the ORDER of operations is also the same. This is the difference between an experiment you can reproduce and one you cannot.

13.2 Component 9 - the training loop

```

1  #define RIGL_INTERVAL 100          /* update the mask every 100 steps */
2
3  size_t step = 0;
4  for (size_t epoch = 0; epoch < epochs; epoch++) {
5
6      for (size_t i = 0; i < trainSize; i++) {
```

```

8      batch_t *b = loader.getBatch(&loader, i);
9
10     forward(model, b->input);
11     backward(model, b->label);
12
13     if (step % RIGL_INTERVAL == 0)
14         rigLStep(model, nLayers, 0.9f, step, totalSteps);
15
16     optimizerFunctions[SGD_M].step(&optim);
17     step++;
18 }
19
20 lrSchedulerStep(&sched);
21 }

```

Listing 11.2 - The training loop, from section 39.9.

Line	Code	What it does and why
1	#define RIGL_INTERVAL 100	T from the algorithm. Over a 10,350-step run this gives 103 mask updates. Too small and the network never settles between swaps; too large and the connectivity barely explores.
10-11	forward(); backward();	The gradients that rigLStep() will read are produced here. The backward pass must produce gradients for INACTIVE weights too - see Chapter 8 - or the GROW step has nothing to rank.
13-14	if (step % RIGL_INTERVAL == 0) rigLStep(...)	The mask update, placed AFTER backward and BEFORE the optimiser step.
16	optimizerFunctions[SGD_M].step(&optim)	The mask-aware update from Component 5. Because it runs after rigLStep(), the weights just grown get their first update in this very iteration rather than waiting a full step.
21	lrSchedulerStep(&sched)	Learning-rate decay, once per epoch, independent of the alpha schedule inside rigLStep().

THE SOURCE CONTRADICTS ITSELF ABOUT THIS ORDERING

Section 37.4 states: 'rigLStep() must be called AFTER the optimizer step but BEFORE the next forward pass. The order matters.' Section 39.9 states the opposite: 'Run rigLStep() BEFORE the optimizer step. If rigLStep() runs AFTER the optimizer, the new weights miss their first update.' The code in 39.9 implements the second. Both cannot be right, and the difference is observable: it decides whether a grown weight waits 0 or 100 steps for its first gradient, and whether the gradients rigLStep() reads have already been zeroed by the optimiser (Component 5 line 16 clears them). BEFORE the optimiser is the correct choice, for exactly the reason 39.9 gives, and because the optimiser would otherwise have wiped the inactive gradients that GROW depends on. Chapter 17, defect D4.

13.3 Component 10 - the two schedules

The learning-rate schedule and the alpha schedule are separate cosines that should share a horizon.

```

1  cosineAnnealingLrInit(&sched, &optimizer,
2                          0.01f,          /* eta_max */
3                          totalEpochs,
4                          1e-5f);         /* eta_min */
5
6  /* alpha(step) = (0.3/2) * (1 + cos(pi * step / T_end)) */
7  /* with T_end = 0.8 * totalSteps */

```

Listing 11.3 - Learning rate and alpha, from section 14.4.

Step	Epoch	alpha	K (of 7,373 active)	Learning rate	Phase
0	0	0.300	2,212	0.010	Explore both values and connectivity
4,140	20	0.150	1,106	~0.006	Consolidate

Step	Epoch	alpha	K (of 7,373 active)	Learning rate	Phase
8,280	40	0.000	0	~0.001	Mask frozen
10,350	50	0.000	0	1e-5	Refine values only

PASS T_END, NOT TOTALSTEPS

`rigLStep()` computes `prog = step / totalSteps` and reaches `alpha = 0` only when `prog = 1`, i.e. at the very last step. The schedule the source specifies wants `alpha = 0` at 80% of training. So the caller must pass `0.8f * totalSteps` as the `totalSteps` argument - the parameter is misnamed, and passing the true total leaves the mask swapping into the final epoch, which is precisely what the schedule exists to prevent. Chapter 17, defect D9.

14. Verifying the Implementation

Each component has a test that isolates it. Run them in build order; a failure at step k with steps $1..k-1$ passing localises the bug to one function.

14.1 Unit tests, component by component

Component	Test	Expected
1 - kth smallest	Mask with 5 known active weights [0.01,0.05,0.03,0.12,0.08], $K=2$	Returns 0.05. Then count how many satisfy ≤ 0.05 : you should get 3, which demonstrates defect D1
1 - guard	$K = \text{count}$	Returns 1e38, and every active weight drops
2 - kth largest	Inactive gradients [0.5,0.1,0.9,0.3], $K=1$	Returns 0.5
2 - guard	$K = \text{count}$	Returns 0.0, and every inactive weight grows
3 - default	Build a model with no sparse specs	Every weightMask is NULL; all existing tests still pass unchanged
4 - matmul	Dense vs 90%-masked forward on identical inputs	Instruction counter falls from 73,728 to about 7,373; outputs differ only in the masked contributions
4 - equivalence	Zero the inactive weights, then run the DENSE matmul	Bit-identical to the masked result - this is the strongest single check that your flat index is right
5 - SGD	One step with 90% of the mask clear	Every inactive weight is exactly 0.0f afterwards, and its gradient is 0.0f
6 - rigLStep	Count active before and after one call	Equal, or differing only by the tie effects of D1
6 - schedule	Call at step 0, $T_{\text{end}}/2$ and T_{end}	$\alpha = 0.30, 0.15, 0.00$ and K scales with it
7 - round trip	serializeSparsity then deserializeSparsity	Every one of the 73,728 bits matches

14.2 Integration checks during a real run

Log these every epoch and watch their shape

- ☐ Active weight count per sparse layer. It must be FLAT. A downward drift means GROW is finding fewer candidates than DROP removes - usually because inactive gradients are zero (defect D3).
- ☐ Number of mask bits that changed since the last RigL step. It should start near K and fall to zero as α decays. If it is zero from the beginning, K is rounding to zero or the mask is NULL.
- ☐ Fraction of swaps that are re-grows of weights dropped in the same step. Should be small; a large fraction is defect D2.
- ☐ Training loss around each RigL step. A spike is expected; failure to recover before the next step means α is too high or T too small.
- ☐ Matmul instruction count. Should stay near $(1 - \text{sparsity}) \times \text{dense}$ for the whole run.
- ☐ Peak heap usage, if you kept the malloc approach. This is where the run dies on the target.

14.3 Results of a reference simulation

The consolidated code in Appendix A was compiled against stub headers and run for 8,200 steps on a synthetic 8,192-weight layer initialised at 10% density, with fresh random dense gradients supplied at each RigL step. This exercises the mask dynamics only - there is no forward pass, no loss and no optimiser - so it says nothing about accuracy. What it does test is every invariant this document claims.

Check	Result
Active count at initialisation	828 of 8,192 (10.1%) - binomial scatter around the requested 819, as expected
Active count after every one of 83 mask updates	828. Conservation held exactly at every step
Inactive weights holding a non-zero value at the end	0
alpha at steps 0 / 4,000 / 7,600	0.3000 / 0.1500 / 0.0018 - the cosine schedule
Behaviour after tEnd = 8,000	K = 0 and the mask stops changing

ONE FINDING THE SIMULATION SURFACED

The DROP step consistently removed 249 weights while K fell from 248 to 1. The cause is instructive: grown weights are set to exactly 0.0, and in this simulation nothing ever moves them, so after the first update hundreds of active weights share the identical magnitude 0.0. No threshold can separate a tied group, so any threshold at or above zero takes all of them. In a real run the optimiser moves grown weights off zero within one step and the ties disappear - but this is exactly the tie pathology behind defect D1, made visible. It is also why the Appendix A code grows **dropped** weights rather than **K**: whatever DROP actually removed, GROW restores the same number, so conservation survives a tie that the K-based version would not.

THE ONE TEST THAT CATCHES THE MOST

Zero every inactive weight, then run the ORDINARY dense matmul and compare against your masked matmul bit for bit. They must agree exactly - masking and zeroing are mathematically identical operations. This single test validates the flat-index computation, the mask plumbing, and the bit-packing convention all at once, and it is the test that catches the silent index-misalignment failure that everything else lets through.

15. What You Will See in the Logs

Every defect in this document has a signature in the training log. Knowing what healthy output looks like means you can spot a broken implementation within the first few hundred steps instead of after a 50-epoch run.

15.1 Healthy output

```

1 [RIGL] rigLStep: layer=0 step=0    alpha=0.3000 K=2212 active=7412
2 [RIGL]   drop thresh=2.481e-03 dropped=2212
3 [RIGL]   grow thresh=8.113e-04 grown=2212
4 [RIGL] rigLStep: layer=0 step=100  alpha=0.2999 K=2211 active=7412
5 [RIGL]   drop thresh=2.106e-03 dropped=2211
6 [RIGL]   grow thresh=7.984e-04 grown=2211
7 ...
8 [RIGL] rigLStep: layer=0 step=4100 alpha=0.1500 K=1111 active=7412
9 [RIGL]   drop thresh=9.02e-04 dropped=1111
10 [RIGL]  grow thresh=1.219e-03 grown=1111
11 ...
12 [RIGL] rigLStep: layer=0 step=8200 alpha=0.0000 K=0    active=7412
13 (no further output - K is zero, the mask is frozen)

```

Listing 13.1 - What a correct run looks like.

Signal	Healthy behaviour	Why
active	Identical on every line, for the whole run	Conservation: K dropped = K grown
alpha	Starts at 0.3000, decays smoothly, reaches 0 at tEnd	The cosine schedule
K	Tracks alpha x active, falls to 0	$K = \text{floor}(\alpha * \text{active})$
dropped and grown	Equal to each other on every line	If they differ, sparsity is drifting
drop thresh	Small and slowly FALLING	As training proceeds the surviving weights grow, so the K-th smallest gets smaller
grow thresh	Small and slowly RISING	Gradients on inactive weights shrink as the model converges, so the bar for being interesting rises relative to them

15.2 Failure signatures

What you see	Almost certainly	Where to look
grow thresh=0.000000 on every line	Defect D3. The inactive gradients are all zero, so the threshold collapses and GROW is picking by scan order. RigL is not running	Is the weight-gradient matmul masked? Is rigLStep() after the optimiser?
active falls a little on every RigL step	DROP is removing more than GROW restores - usually the same D3, with too few gradients above the threshold	Compare dropped and grown on each line
active rises over time	GROW is over-matching, typically many gradients tied at exactly the threshold	The tie handling of defect D1
dropped and grown differ by 1 consistently	Defect D1, the off-by-one from non-strict comparisons	The $\leq$ and $\geq$ in rigLStep()
dropped is constant while K falls	Many active weights share an identical magnitude - usually 0.0 because grown weights were never updated	Is the optimiser running? Is it masked correctly?
No [RIGL] lines at all	Every weightMask is NULL, so rigLStep() skips every layer	Did buildSequentialModel() get sparse=true ?

What you see	Almost certainly	Where to look
K=0 from the very first step	alpha or numActive is zero - a mask filled with the wrong probability, or <code>bernoulliFillMask(mask, 0.9)</code> when you meant 0.1	The P(active) inversion in Chapter 13
Loss spikes at every RigL step and never recovers	alpha too high or the interval too short - the network cannot re-learn 2,000 connections in 100 steps	Lower alphaInit, or raise RIGL_INTERVAL
Instruction counter unchanged after masking	The mask is not reaching the matmul kernel	A wrapper is still passing NULL
Hard fault or malloc failure a few steps in	Defect D8 - 259 KB of scratch heap on a 320 KB device	Build the histogram variant

15.3 The three numbers to plot

- **Active count per layer, against step.** Should be a flat line. This single plot catches D1, D3 and most plumbing mistakes.
- **Mask churn** - how many bits changed since the last RigL step - against step. Should start near K and decay to zero with alpha. A flat-zero churn means the mask is not moving; a churn that stays high after tEnd means you passed totalSteps instead of 0.8 x totalSteps (defect D9).
- **Training loss, with the RigL steps marked.** You should see a small spike at each swap that recovers within a few dozen steps, and the spikes should shrink as alpha decays. Spikes that grow over time mean the swaps are destroying more than they gain.

ADD ONE ASSERTION AND MOST OF THIS BECOMES UNNECESSARY

The consolidated code in Appendix A logs an error whenever `grown != dropped`. That one check turns the silent, slow-motion failure of sparsity drift into a loud message on the step it first happens. If you take nothing else from this chapter, take that assertion.

16. An Implementation Plan

Ten components, ordered so that each one is testable before the next depends on it. The estimates assume familiarity with the ODT codebase and no surprises; they are there to show the SHAPE of the work, not to be held to.

Order	Task	Chapter	Rough effort	Done when
1	Read the prerequisites and the hand-worked step	2, 3	1 hour	You can predict the DROP and GROW sets of the toy layer without running anything
2	findAbsKthSmallestActive + unit test	5	2 hours	The toy example returns 0.30, and the K >= count guard returns 1e38
3	findAbsKthLargestInactive + unit test	6	1 hour	Returns 0.40 on the toy example; the guard returns 0.0
4	Decide exact vs histogram K-selection	6, 17	1 hour	You have measured the scratch allocation your largest layer would need and compared it against free SRAM
5	weightMask field + NULL default	7	30 minutes	The whole existing test suite still passes unchanged
6	Mask-aware matmul + the zero-equivalence test	8	3 hours	Masked output is bit-identical to dense-with-zeros, and the instruction counter drops to ~10%
7	Mask-aware SGD (and AdamW if used)	9, 10	2 hours	After one step, every inactive weight is exactly 0.0f
8	rigLStep() itself	11	4 hours	Active count is conserved across 100 simulated steps
9	serializeSparsity + deserializeSparsity	12	2 hours	A save/load round trip reproduces all 73,728 bits
10	UserAPI mask creation	13	1 hour	A model built with sparse=true logs the expected active count
11	Training-loop integration	13	1 hour	The [RIGL] lines of Chapter 15 appear, with dropped == grown
12	First real run + the three plots	15	1 day	Flat active count, decaying churn, recovering loss spikes

16.1 Checkpoints where you should stop and verify

Do not proceed past these until they hold

- ☐ After step 6: masked matmul equals dense-with-zeros, bit for bit. If this fails, everything downstream is built on a wrong index.
- ☐ After step 7: inactive weights are exactly 0.0f after an optimiser step - not merely small.
- ☐ After step 8: 100 simulated rigLStep() calls leave the active count unchanged, and `grown == dropped` never logs an error.
- ☐ After step 11: `grow_thresh` is strictly positive on every line. A zero there means defect D3 and everything above it was wasted effort.

THE ORDER IS NOT ARBITRARY

Each step is verifiable using only the steps before it. Building rigLStep() first - the tempting move, since it is the interesting part - means its first test depends on four unverified components at once, and a failure tells you nothing about where the fault is. The K-selection functions are pure, take no ODT state, and can be tested with eight numbers in an array; start there.

17. Ten Defects Found While Consolidating

These are not criticisms of the source, which is a careful and unusually complete piece of documentation. They are what surfaces when fragments written in 39 separate chapters are placed side by side: inconsistencies that are invisible locally and obvious globally. Each entry gives the evidence, the consequence, and a fix.

17.1 Severity summary

ID	Defect	Severity	Fix cost
D1	DROP/GROW thresholds swap K+1 weights, not K	Low	1 line
D2	Just-dropped weights can be regrown in the same step	Medium	~6 lines
D3	GROW depends on gradients that may never be computed	Critical	Design decision
D4	Two sections give opposite call ordering	High	Doc fix
D5	Momentum buffer not cleared on regrowth	Medium	1 line
D6	AdamW memory figure counts only active weights	High	Re-plan the model
D7	Two incompatible linearConfig_t definitions	Low	Doc fix
D8	malloc of 259 KB on a 320 KB device	Critical	~15 lines
D9	T_end vs totalSteps; format version not bumped	Medium	2 lines
D10	Complexity estimate uses n where count is correct	Low	Doc fix

17.2 D1 - The thresholds are off by one

Evidence. Section 39.2's own worked example: active magnitudes [0.01, 0.05, 0.03, 0.12, 0.08] with $K = 2$ give $\text{thresh} = \text{vals}[2] = 0.05$, and the text claims 'exactly 2 weights dropped'. The condition is $|w| \leq 0.05$, which matches 0.01, 0.03 **and** 0.05 - three weights.

Consequence. Every DROP removes $K+1$ and every GROW adds $K+1$, so the active count is preserved by cancellation. The swap is one larger than requested, and where magnitudes tie - which happens constantly, because newly grown weights are all exactly 0.0 - the cancellation is no longer exact and sparsity drifts.

Fix. Make the comparison strict on one side:

```

1  /* DROP: drop exactly K */
2  if (tensorBoolGet(mask,i) && fabsf(w[i]) < dropThresh) { ... }
3
4  /* or keep <= and take the (K-1)-th value instead: */
5  float dropThresh = findAbsKthSmallestActive(weights, mask, K - 1);

```

Listing 12.1 - Two equivalent one-line fixes for D1.

17.3 D2 - Drop and grow are not disjoint

Evidence. In Listing 9.1, `findAbsKthLargestInactive()` on line 39 runs after the DROP loop has already cleared mask bits. The just-dropped weights are inactive by then, so they compete for growth.

Consequence. A weight with a small value but a steep gradient is dropped and immediately regrown - its trained value destroyed for nothing, and part of the swap budget wasted. The RigL paper draws the grow candidates from the set that was inactive **before** the drop.

Fix. Snapshot the mask before dropping, and require a grow candidate to have been inactive in the snapshot:

```

1  /* before the DROP loop */
2  tensor_t *prevMask = cloneBoolTensor(mask);
3
4  /* in the GROW loop, add one condition */

```

```

5   if (!tensorBoolGet(mask, i) &&
6       !tensorBoolGet(prevMask, i) &&          /* was ALREADY inactive */
7       fabsf(g[i]) >= growThresh) { ... }
8
9   freeBoolTensor(prevMask);

```

Listing 12.2 - Excluding just-dropped weights. Costs one extra 9 KB mask per sparse layer, transiently.

17.4 D3 - The gradients GROW needs may not exist

THIS IS THE DEFECT THAT SILENTLY TURNS RIGL INTO STATIC SPARSITY

GROW ranks inactive weights by $|\text{gradient}|$. Three separate parts of this implementation can leave those gradients at zero. (a) If the weight-gradient matmul is masked like the forward matmul (Chapter 8), inactive weights never receive a gradient at all. (b) Component 5 (Chapter 9) line 16 sets `grad[i] = 0.0f` for inactive weights on EVERY optimiser step. (c) `rigLStep()` itself zeroes them at the end of every call. If any of these wins the race, `findAbsKthLargestInactive()` sees all zeros, returns 0.0, and the GROW condition $|g| \geq 0$ matches every inactive weight - so the first K+1 encountered in index order are activated. The mask then evolves by scan order, not by gradient. Training still converges, sparsity still looks right, and RigL is not actually running.

Fix. Three things must all hold: the weight-gradient matmul stays dense; `rigLStep()` runs before the optimiser clears anything (D4); and the gradients `rigLStep()` reads are the ones from the backward pass of the same iteration. **Verify it** by asserting that `findAbsKthLargestInactive()` returns a strictly positive threshold - if it ever returns 0.0 on a layer with more inactive weights than K, you have this bug.

17.5 D4 - Contradictory ordering instructions

Section	Says	Reason given
37.4	AFTER the optimizer step, before the next forward	'This ensures that the gradient used for GROW reflects the current weight update'
39.9	BEFORE the optimizer step	'This ensures that newly grown weights ($w=0$) get their first gradient update in the same step they are activated'

Resolution. 39.9 is correct and its code implements it. Beyond the argument it gives, running after the optimiser would mean the inactive gradients had already been zeroed by Component 5 - straight into defect D3. Section 37.4 should be corrected.

17.6 D5 - Stale momentum on regrowth

Section 13.4 identifies stale Adam moments as a hazard and clears `m` and `v`. SGD with momentum has the same problem: the velocity buffer entry for a regrown weight still holds the velocity it had before it was dropped, so its first update is a jump in a stale direction. Clear it in the same branch that zeroes the weight.

17.7 D6 - The AdamW memory figure is not achievable

Evidence. Section 13.4: '7,373 active weights x 3 tensors x 4 bytes = 88 KB'. That arithmetic is right; the premise is not. `m` and `v` are allocated with the same dense shape as the parameters, so the real figure counts 73,728 elements, not 7,373: four dense float tensors plus the mask is 1,188,864 bytes for this one layer, on a device with 320 KB of SRAM.

Fix. Either store the moments sparsely - a real change, since it breaks O(1) indexing - or accept SGD, or shrink the layer. The source itself points at the third option in 39.10.

17.8 D7 - Two struct definitions

Sections 11.4 and 39.4 both present `linearConfig_t` with different field lists (Chapter 7 of this document tabulates them). Retyping either one will break the build. Edit the real header instead.

17.9 D8 - malloc on the target

`findAbsKthLargestInactive()` allocates `count * sizeof(float)` where `count` is the number of INACTIVE weights - 66,355 for the HAR layer, so 259 KB of a 320 KB device, transiently, every 100 steps, while everything else is resident. It will not allocate; and if a smaller layer lets it succeed, repeated allocation and release of large differently-sized blocks fragments the heap.

Fix. Either allocate one scratch buffer at initialisation sized for the largest sparse layer and reuse it, or replace the sort with a histogram:

```

1  /* Histogram threshold: O(n) time, O(1) memory, no allocation. */
2  #define NBINS 256
3  uint16_t hist[NBINS] = {0};
4
5  /* pass 1: find the maximum magnitude among inactive weights */
6  float gmax = 0.0f;
7  for (size_t i = 0; i < n; i++)
8      if (!tensorBoolGet(mask,i) && fabsf(g[i]) > gmax) gmax = fabsf(g[i]);
9
10 /* pass 2: bin the magnitudes */
11 float scale = (float)(NBINS - 1) / (gmax > 0.0f ? gmax : 1.0f);
12 for (size_t i = 0; i < n; i++)
13     if (!tensorBoolGet(mask,i))
14         hist[(size_t)(fabsf(g[i]) * scale)]++;
15
16 /* pass 3: walk down from the top bin until K are accounted for */
17 size_t acc = 0;
18 int b = NBINS - 1;
19 for (; b >= 0; b--) {
20     acc += hist[b];
21     if (acc >= K) break;
22 }
23 float thresh = (float)b / scale; /* approximate to one bin width */

```

Listing 12.3 - A fixed-memory replacement for the selection sort. 512 bytes of histogram instead of 259 KB of heap, and $O(n)$ instead of $O(n \cdot K)$.

IS AN APPROXIMATE THRESHOLD ACCEPTABLE?

Yes, and this is worth being explicit about. The threshold decides which connections to try next; it is re-computed 103 times over a run, and a weight that misses the cut this time will be considered again in 100 steps. Being within one bin width of the exact K-th value changes which few borderline weights are grown, not whether the algorithm works. Exactness here buys nothing and costs 259 KB.

17.10 D9 - Two smaller correctness gaps

- **T_end.** `rigLStep()` reaches $\alpha = 0$ only at `step == totalSteps`, but the schedule in section 2.3 specifies $T_{\text{end}} = 0.8 \times \text{total}$. The caller must pass `0.8f * totalSteps`, or the mask keeps swapping into the final epoch. Better: give `rigLStep()` an explicit `tEnd` parameter instead of overloading `totalSteps`.
- **Format version.** Adding the sparsity presence byte changes every tensor record, including in dense checkpoints, while chapter 17 of the SOURCE calls the format 'locked v2'. Bump to v3 and have the reader accept both.

17.11 D10 - The complexity estimate uses the wrong n

Section 7.4 gives $n * K = 8192 \times 819 = 6.7 \text{ M}$ comparisons. The sort runs over the gathered ACTIVE values, so the cost is `count * K`. At 90% sparsity that is ten times smaller for the DROP side - and, importantly, nine times LARGER for the GROW side, where `count` is the inactive population. The corrected figures are in Chapters 5 and 6.

WHAT TO DO WITH THIS LIST

Fix D3, D8 and D4 before you run anything - they are the difference between RigL and an expensive random mask. Fix D1, D2 and D5 before you trust an accuracy number. D6 changes what model you can train at all, so settle it while the architecture is still on paper. D7, D9 and D10 are documentation and hygiene, and cost minutes.

18. Common Confusions

Is RigL pruning?

No, and the distinction matters. Pruning trains a dense network and then removes weights, so you pay for dense training first. RigL is sparse from step one and stays at a fixed sparsity for the whole run - the memory and forward-pass compute never exceed the sparse budget. What it learns, in addition to the weight values, is WHICH connections should exist.

If 90% of weights are skipped, why is training not 10x faster?

Because only the forward pass and the loss-propagation matmul are masked. The weight-gradient matmul must stay dense, or the GROW step has no signal to rank inactive weights by (Chapter 8, defect D3). Roughly speaking you save on two of the three matmuls, so the end-to-end training speedup is well short of the sparsity ratio. **Inference**, where only the forward pass runs, does get close to the full benefit - which is the case that matters on the device.

Does the model get smaller?

Not as implemented here. The weight tensor keeps its full dense allocation and simply contains zeros at inactive positions, so a 73,728-weight layer still occupies 288 KB plus a 9 KB mask. Getting the memory back requires a compressed format such as CSR, which costs index storage and destroys the O(1) random access the masked matmul relies on. What RigL buys here is compute and energy, not footprint - be precise about that when reporting results.

Why not just keep the largest weights and stop swapping?

That is magnitude pruning with a fixed mask, and it is a reasonable baseline - worth measuring, in fact. The problem is that the best connectivity early in training is not the best connectivity later: a connection that looks useless at step 500 may be exactly what the network needs at step 5,000, and a fixed mask can never discover it. The gradient of an inactive weight is the only signal that can tell you this, and using it is the whole contribution of RigL.

What happens if I never call `rigLStep()`?

You get static random sparsity: the Bernoulli mask from initialisation, frozen. The model still trains and will reach a noticeably worse accuracy than either dense or RigL. This is also, unfortunately, what defect D3 silently degrades to - which is why the `grow_thresh` line in your log deserves attention.

Why every 100 steps and not every step?

Two reasons. Cost: a RigL step is O(n) or worse per layer and would be a significant fraction of the per-step budget if run every iteration. Stability: a grown weight starts at zero and needs time to become useful; swapping every step means nothing ever settles. The interval is a hyperparameter - 100 is the source's choice and a sensible default, and it interacts with alpha, since what matters is how many connections move per unit of training time.

Can I use RigL on convolutional layers?

In principle yes - section 20.4 of the source sketches a `kernelMask` for `conv1dConfig_t` with the same treatment. In practice the consolidated `rigLStep()` skips anything that is not LINEAR, so conv support means extending both the layer loop and the conv inner loop. For the HAR model it also matters less: a 16 x 9 x 3 kernel has 432 weights against the linear layer's 73,728, so the linear layers are where the compute is.

Should the last layer be sparse?

No. The 64 x 6 classifier is 0.5% of the model's weights, so sparsifying it saves nothing measurable, and removing connections there removes them from individual output classes - the place where damage is most visible. The same argument covers biases and normalisation parameters. Leaving the first and last layers dense is standard practice in the pruning literature and costs you almost nothing in compression.

What is the difference between sparsity and the mask?

Sparsity is a number you choose - 0.9 - and it is fixed for the run. The mask is the specific set of active positions realising that number, and it changes every 100 steps. Note that `rigLStep()` takes a `sparsityTarget` argument in the source and never uses it: the sparsity is whatever the initial `bernoulliFillMask()` created, and `rigLStep` only preserves it.

My active count is 7,412 but I asked for 7,373. Is that a bug?

No. `bernoulliFillMask` makes an independent draw per weight, so the realised count is binomial: for 73,728 weights at $p = 0.1$ the mean is 7,373 with a standard deviation of about 81. A count within a few hundred of the target is expected. If you need exactly the target, shuffle an array with exactly that many ones instead of drawing independently.

Do I need FQT and quantization working before RigL?

No, and it is easier if you do not. Everything in this document assumes FLOAT32 weights and gradients - the direct `(float*)t->data` casts are only valid in that case. Get RigL correct in float first; extending the threshold comparisons to SYM_INT32 mantissas, which requires reconstructing `mantissa * scale` before comparing magnitudes, is a separate and clearly-scoped piece of work.

19. Appendix - Consolidated RigL.h and RigL.c

Everything above, assembled into the two new files the implementation adds. The other components are edits to existing files and are listed in the chapters that describe them. This listing is the source's code with the D1, D2 and D9 fixes applied, and each fix marked; it is offered as a starting point, not as tested software.

19.1 RigL.h

```

1  #ifndef RIGL_H
2  #define RIGL_H
3
4  #include <stddef.h>
5  #include "Tensor.h"
6  #include "Layer.h"
7
8  /* K-selection helpers (implemented in MinMax.c) */
9  float findAbsKthSmallestActive (tensor_t *weights, tensor_t *mask,
10                                size_t K);
11  float findAbsKthLargestInactive(tensor_t *grads, tensor_t *mask,
12                                size_t K);
13
14  /* One RigL mask update over every sparse LINEAR layer of the model.
15   * Call AFTER backward() and BEFORE the optimiser step.
16   * tEnd: the step at which alpha reaches zero (typically 0.8*total). */
17  void rigLStep(layer_t **model, size_t numLayers,
18               float alphaInit, size_t step, size_t tEnd);
19
20 #endif /* RIGL_H */

```

Listing A.1 - RigL.h.

19.2 RigL.c

```

1  #define SOURCE_FILE "RIGL"
2
3  #include "Common.h"
4  #include "RigL.h"
5  #include "Tensor.h"
6  #include "MinMax.h"
7  #include "Layer.h"
8  #include "Linear.h"
9  #include <math.h>      /* cosf, fabsf */
10
11  void rigLStep(layer_t **model, size_t numLayers,
12               float alphaInit, size_t step, size_t tEnd) {
13
14      /* --- alpha schedule; frozen once step passes tEnd (fix D9) --- */
15      if (tEnd == 0) return;
16      if (step >= tEnd) return;
17      float prog = (float)step / (float)tEnd;
18      float alpha = 0.5f * alphaInit * (1.0f + cosf(3.14159265f * prog));
19
20      for (size_t l = 0; l < numLayers; l++) {
21
22          if (model[l]->type != LINEAR) continue;
23
24          linearConfig_t *cfg = model[l]->config->linear;
25          tensor_t *mask = cfg->weightMask;
26          if (mask == NULL) continue;
27

```

```

28     tensor_t *weights = cfg->weights->param;
29     tensor_t *grads    = cfg->weights->grad;
30     size_t n = calcNumberOfElementsByTensor(weights);
31     float *w = (float *)weights->data;
32     float *g = (float *)grads->data;
33
34     size_t numActive = 0;
35     for (size_t i = 0; i < n; i++)
36         if (tensorBoolGet(mask, i)) numActive++;
37
38     size_t K = (size_t)(alpha * (float)numActive);
39     if (K == 0) continue;
40
41     PRINT_INFO("rigLStep: layer=%zu step=%zu alpha=%.4f K=%zu",
42               l, step, alpha, K);
43
44     /* snapshot of the pre-drop mask (fix D2) */
45     tensor_t *prevMask = cloneBoolTensor(mask);
46
47     /* ---- DROP: strict < drops exactly K (fix D1) ---- */
48     float dropThresh = findAbsKthSmallestActive(weights, mask, K);
49     size_t dropped = 0;
50     for (size_t i = 0; i < n; i++) {
51         if (tensorBoolGet(mask, i) && fabsf(w[i]) < dropThresh) {
52             tensorBoolSet(mask, i, false);
53             w[i] = 0.0f;
54             dropped++;
55         }
56     }
57
58     /* ---- GROW: only weights inactive BEFORE the drop (D2) ---- */
59     float growThresh = findAbsKthLargestInactive(grads, prevMask,
60                                                  dropped);
61     size_t grown = 0;
62     for (size_t i = 0; i < n && grown < dropped; i++) {
63         if (!tensorBoolGet(prevMask, i) &&
64             !tensorBoolGet(mask, i) &&
65             fabsf(g[i]) > growThresh) {
66             tensorBoolSet(mask, i, true);
67             w[i] = 0.0f;          /* grown connections start at 0 */
68             grown++;
69         }
70     }
71
72     if (grown != dropped)
73         PRINT_ERROR("rigLStep: layer=%zu dropped=%zu grown=%zu",
74                   l, dropped, grown);
75
76     freeBoolTensor(prevMask);
77
78     /* ---- clear gradients of everything still inactive ---- */
79     for (size_t i = 0; i < n; i++)
80         if (!tensorBoolGet(mask, i)) g[i] = 0.0f;
81 }
82 }

```

Listing A.2 - RigL.c with the D1, D2 and D9 fixes applied and marked. The **grown != dropped** check is the runtime assertion of the conservation property - if it ever fires, sparsity is drifting.

WHAT THIS LISTING IS AND IS NOT

It is a faithful consolidation of the source's algorithm with three specific defects repaired and the repairs labelled. It has not been compiled against the ODT headers, and it assumes helpers - `cloneBoolTensor`, `freeBoolTensor` - that the source does not document and you may have to write. It still uses the malloc-based K-selection, so defect D8 remains open; apply Listing 12.3 before putting this on the target.

20. Appendix - Glossary

Every term used in this document, in one place. ODT-specific terms are marked (ODT); the rest are general to sparse training.

Term	Meaning
active weight	A weight whose mask bit is 1. It participates in the forward pass and is updated by the optimiser.
alpha, alpha(t)	The fraction of active weights swapped at a RigL step. Decays from <code>alphaInit</code> (0.3) to zero on a cosine schedule.
alphaInit	The starting value of alpha. 0.3 in the source, meaning 30% of connections move at the first swap.
bernoulliFillMask	(ODT) Fills a BOOL tensor with independent Bernoulli draws. Its argument is <code>P(ACTIVE)</code> , so 90% sparsity means passing 0.1.
bit packing	Storing one boolean per bit rather than per byte. A 73,728-element mask occupies 9,216 bytes.
BOOL tensor	(ODT) A tensor whose quantization type is BOOL; the storage format for a mask.
conservation	The RigL invariant that K weights dropped equals K weights grown, so the active count never changes.
CSR	Compressed Sparse Row - a format that stores only non-zeros plus indices. Saves memory, loses $O(1)$ random access; not used here.
dense	The ordinary, unmasked case. In ODT a layer is dense exactly when its <code>weightMask</code> is NULL.
DROP	The half of a RigL step that deactivates the K active weights with the smallest $ w $.
executeOp	(ODT) The universal funnel through which kernels are invoked, with prologue, kernel and epilogue stages.
flat index	The one-dimensional position of an element in a row-major tensor: $\text{row} * \text{columnsPerRow} + \text{column}$. The mask, the weight and its gradient all share it.
FQT	Fully Quantized Training - the integer-arithmetic training path in ODT, in which weights and gradients may be <code>SYM_INT32</code> rather than <code>FLOAT32</code> .
GROW	The half of a RigL step that activates the K inactive weights with the largest $ \text{gradient} $.
HAR	Human Activity Recognition - the target task: classifying activities from wrist-worn IMU signals.
inactive weight	A weight whose mask bit is 0. Skipped in the forward pass, forced to exactly 0.0 by the optimiser, but still receiving a gradient - which is what makes GROW possible.
K	The number of weights swapped at one RigL step: $\text{floor}(\alpha * \text{numActive})$.
K-selection	Finding the K-th smallest or largest value in a set. Implemented here by partial selection sort or by histogram.
kernelMask	(ODT, proposed) The Conv1d analogue of <code>weightMask</code> ; described in section 20.4 of the source, not implemented.
layerConfig_t	(ODT) A union of per-layer-type configs. Reading <code>.linear</code> on a non-LINEAR layer yields garbage, which is why <code>rigLStep()</code> checks the type first.
magnitude pruning	Removing the smallest-magnitude weights. RigL's DROP step, but without any regrowth.
mask	The BOOL tensor recording which weights are active. One bit per weight, same element count as the weight tensor.
mask churn	How many mask bits changed at a RigL step. Should track K and decay to zero.
MCU	Microcontroller unit - here an STM32 Nucleo-F746ZG: Cortex-M7 at 216 MHz with 320 KB of SRAM and no MMU.
parameter_t	(ODT) A weight tensor paired with its gradient tensor.
partial selection sort	Selection sort stopped after K+1 positions. $O(\text{count} * K)$ rather than $O(\text{count}^2)$.

Term	Meaning
PPCA replay	(ODT) Probabilistic-PCA-based continual learning in the library; unrelated to RigL, and its chapter confirms it needs no changes.
RigL	Rigging the Lottery: sparse training in which connectivity is learned by dropping low-magnitude weights and growing high-gradient ones.
rigLStep	The function that performs one DROP/GROW update over every sparse layer of a model.
SR	Stochastic rounding - rounding up or down at random with probability proportional to distance, used in ODT's integer training path.
sparsity, s	The fraction of weights that are inactive. 0.9 means nine in ten are off.
straight-through estimator	Treating a non-differentiable forward operation as the identity in the backward pass; used in quantization, not needed by RigL.
SYM_INT32	(ODT) A symmetric integer tensor type storing int32 mantissas plus a scale. Weight and gradient tensors may use it under FQT, which invalidates the direct float* casts in this document.
tEnd	The step at which alpha reaches zero and the mask freezes. Should be 0.8 x totalSteps, not totalSteps.
tensor_t	(ODT) The universal container: shape, quantization descriptor and a flat byte buffer.
tensorBoolGet / Set	(ODT) Read or write one mask bit by flat index.
weightMask	(ODT) The field added to linearConfig_t holding the layer's mask, or NULL for a dense layer.
zeroInactiveWeights	(ODT) Enforces the invariant that every inactive weight is exactly 0.0; called once at initialisation.

21. Appendix - Where Every Fragment Came From

The full extraction map, so any statement in this document can be traced back to the source PDF.

Source section	Chapter of the source	Content	Used in
2.1	Part 2 intro	Why sparse training; drop/grow rationale	Ch 1
2.2	Part 2 intro	The algorithm in pseudocode	Ch 1, 9
2.3	Part 2 intro	Cosine alpha schedule and LR synergy	Ch 1, 11
2.4	Part 2 intro	Published results; the ODT gap list	Ch 1, 2
1.4	Common.h	SOURCE_FILE pattern, logging for RigL.c	Appendix A
2.4 (ch 2)	DTypes	FLOAT32 vs SYM_INT32 access	Ch 3, 9
7.4	MinMax	Both K-selection functions, complexity	Ch 3, 4, 12
8.4	Matmul	Mask-aware inner loop; counter verification	Ch 6
9.4	RNG/Bernoulli	Mask initialisation and seeding	Ch 11
11.4	Linear	weightMask field; mask memory	Ch 5
12.4	Sgd	Mask-aware SGD kernel	Ch 7
13.4	AdamW	Mask-aware AdamW; moment clearing	Ch 8, 12
14.4	LrScheduler	LR and alpha schedule together	Ch 11
17.5	Serialize	serializeSparsity, deserializeSparsity	Ch 10
20.4	Conv1d	kernelMask extension (not implemented)	Ch 2
33.4	Comparison	DROP/GROW as threshold comparisons	Ch 1
37.4	TrainingLoopApi	Integration point; ordering claim	Ch 11, 12
38.4	UserAPI	Mask creation in buildSequentialModel	Ch 11
39.1-39.10	RigL Complete	All seven components; training loop; expected results	Ch 3-11

Sections 3.4, 4.4, 5.4, 6.4, 10.4, 15.6, 16.4, 18.4, 19.4, 21.4, 22.4, 23.4, 24.4, 25.5, 26.4, 27.4, 28.4, 29.6, 30.4, 31.4, 32.5, 34.5, 35.4 and 36.4 were read and contain no RigL code - they state that the file in question needs no change, which is itself useful information: it means the mask does not have to be threaded through activations, pooling, normalisation, flattening, the data loader, or the PPCA replay path.